

FFT Implementation on FPGA Using Xilinx FFT IP and Zynq PS/PL Integration

Baryonic Space FPGA/Digital Design Engineer Interview Project

7 Mayıs 2026

1 Introduction

Bu çalışma, Baryonic Space FPGA/Digital Design Engineer Interview Project kapsamında gerçekleştirilen Part 1, Part 2, Part 3 ve Part 4 aşamalarını dokümanite etmektedir. Bu aşamaların amacı, Xilinx FFT IP kullanılarak FPGA üzerinde temel bir FFT veri yolunun kurulması, Hamming window entegrasyonu ile spectral leakage etkisinin incelenmesi, frekans domeninde three-term windowing eşdeğerliğinin doğrulanması ve son aşamada bu yapının Zynq PS/PL mimarisi üzerinde AXI-Lite kontrollü gerçek bir sistem bileşeni haline getirilmesidir.

Bu çalışma, FPGA üzerinde DSP algoritması gerçekleştirme sürecinin dört doğrulama adımını kapsamaktadır. Amaç sadece FFT IP'yi çalıştırmak değil; AXI-Stream veri akışını, fixed-point veri paketlemeyi, FFT frekans bileşeni matematiğini, leakage davranışını ve SystemVerilog testbench doğrulamasını birlikte göstermektir. Part 1'de naive FFT veri yolu kurulmuş ve leakage gözlenmiştir. Part 2'de aynı FFT veri yolu korunarak giriş Hamming window bloğu eklenmiş ve leakage'in azaltılması hedeflenmiştir. Part 3'te ise zaman domenindeki pencerelemenin frekans domenindeki üç terimli eşdeğeri ayrı bir donanım bloğu ile gerçekleştirilmiş ve Part 2 ile sonuç tutarlılığı doğrulanmıştır.

Part 1 kapsamında gerçekleştirilen temel adımlar aşağıda özetlenmiştir:

- Xilinx FFT IP çekirdeği 12-bit veri genişliği ile yapılandırılmıştır.
- Vivado tarafından üretilen instantiation kullanılarak FFT IP, Verilog top modüle bağlanmıştır.
- Testbench 200 MHz saat frekansında çalıştırılmıştır.
- Uzunluğu 256 olan iki ardışık kompleks üstel giriş uygulanmıştır: 12.5 MHz ve 10.0 MHz.
- FFT çıkışında Verilog ile $|X[k]|^2$ hesabı gerçekleştirilmiştir.
- FFT IP'nin `xk_index` çıkışı kullanılarak tepe frekans bileşeni konumu tespit edilmiştir.
- 12.5 MHz sinyalinin FFT frekans bileşeni merkezine tam oturması, 10 MHz sinyalinin ise leakage üretmesi beklenmiş ve bu davranış simülasyonda doğrulanmıştır.

Part 2 kapsamında gerçekleştirilen temel adımlar aşağıda özetlenmiştir:

- Part 1’de kurulan Xilinx FFT IP, AXI-Stream, `abs_sq` ve `peak_detector` zinciri korunmuştur.
- FFT girişinden önce zaman alanı Hamming window bloğu eklenmiştir.
- Hamming katsayıları Python’da üretilmiş ve Q1.15 fixed-point formata çevrilmiştir.
- Katsayılar `hamming_q1_15.mem` dosyasına yazılmış ve Verilog’da `$readmemh` ile okunmuştur.
- Her real/imag giriş örneği kendi frame index’indeki Hamming katsayısı ile çarpılmıştır.
- Windowed FFT sonuçları naive FFT sonuçlarıyla karşılaştırılmıştır.
- 10 MHz için uzak yan lobların Part 1’e göre bastırıldığı gösterilmiştir.

Part 3 kapsamında gerçekleştirilen temel adımlar aşağıda özetlenmiştir:

- Part 1’deki FFT çekirdeği korunmuş, pencereleme FFT çıkışına taşınmıştır.
- Hamming window’un frekans domenindeki üç terimli eşdeğeri `three_term.v` ile gerçekleştirilmiştir.
- Her output bin, kendi değeri ile circular komşu binleri kullanılarak yeniden hesaplanmıştır.
- Elde edilen yeni kompleks binler için yine `abs_sq.v` ve `peak_detector.v` zinciri kullanılmıştır.
- Part 3 peak bin ve enerji sonuçları Part 2 ile doğrudan karşılaştırılmıştır.
- Zaman domeni ve frekans domeni pencereleme yaklaşımlarının donanımsal eşdeğerliği doğrulanmıştır.

2 Theoretical Background

Bu çalışmanın amacı, Xilinx FFT IP kullanılarak FPGA üzerinde temel bir DSP akışının gerçekleştirilmesidir. Projenin ilk aşamasında 12-bit genişliğinde Xilinx FFT IP oluşturulmuş, 200 MHz saat frekansı altında iki adet 256 örneklilik kompleks üstel sinyal FFT’ye verilmiş ve FFT çıkışında $|X[k]|^2$ hesaplanarak `pk_index` yardımıyla spektral peak konumu bulunmuştur. Bu bölümde kullanılan DSP yaklaşımı, örnekleme, pencereleme, DTFT, DFT ve spectral leakage kavramları üzerinden açıklanmaktadır.

2.1 Sampling and Windowing Distinction

Bu projedeki en kritik kavramsal ayrımlardan biri, örnekleme ile pencereleme işlemlerinin birbirine karıştırılmamasıdır. Her iki işlem de sinyalin yalnızca belirli bir bölümünü seçiyor gibi görünse de fiziksel ve matematiksel rolleri farklıdır.

Sürekli zamanlı ideal kompleks üstel sinyal

$$x_c(t) = Ae^{j2\pi f_0 t} \quad (1)$$

şeklinde ifade edilir. İdeal örnekleme, bu sinyalin dürtü treni ile çarpılması olarak modellenir:

$$p(t) = \sum_{n=-\infty}^{\infty} \delta(t - nT_s) \quad (2)$$

$$x_s(t) = x_c(t)p(t) \quad (3)$$

$$x_s(t) = \sum_{n=-\infty}^{\infty} x_c(nT_s) \delta(t - nT_s) \quad (4)$$

Buradan elde edilen ayrık zaman dizisi

$$x[n] = x_c(nT_s) \quad (5)$$

olur. Burada

$$T_s = \frac{1}{f_s} \quad (6)$$

olduğundan Python referans modelinde doğrudan

$$x[n] = Ae^{j2\pi f_0 nT_s} = Ae^{j2\pi \frac{f_0}{f_s} n} \quad (7)$$

üretilebilmektedir. Başka bir deyişle Python kodu fiziksel olarak dürtü treni üretmemekte; dürtü treni ile örneklenmiş olsaydı elde edilecek ayrık zaman dizisini doğrudan hesaplamaktadır.

Örnekleme sonrasında elimizde sonsuz uzunluklu bir ayrık zaman dizisi bulunduğu varsayılır. FFT ise sonsuz uzunluklu bir sinyal üzerinde değil, sonlu uzunluklu bir frame üzerinde çalışır. Bu projede FFT uzunluğu

$$N = 256 \quad (8)$$

olarak seçildiği için yalnızca

$$n = 0, 1, \dots, 255 \quad (9)$$

aralığındaki örnekler işleme alınmaktadır. Bu seçim, ayrık zaman alanında rectangular window uygulanması olarak yorumlanır:

$$x_N[n] = x[n]w_R[n] \quad (10)$$

burada rectangular window

$$w_R[n] = \begin{cases} 1, & 0 \leq n \leq N-1 \\ 0, & \text{diğer durumlarda} \end{cases} \quad (11)$$

şeklindedir. Bu projede $N = 256$ olduğundan rectangular window, yalnızca ilk 256 örneği seçmektedir. Özetle, dürtü treni sürekli zamandan ayrık zamana geçişi modellerken; rectangular window ayrık zaman dizisinden FFT frame'inin seçilmesini ifade etmektedir.

2.2 DTFT and DFT Interpretation

Ayrık zamanlı sinyalin sürekli frekans değişkenli dönüşümü DTFT ile tanımlanır:

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega n}$$

Burada frekans değişkeni sürekli ve

$$\omega \in [-\pi, \pi] \quad (12)$$

aralığında değerlendirilir. DTFT ayrıca 2π periyodiktir:

$$X(e^{j(\omega+2\pi)}) = X(e^{j\omega}) \quad (13)$$

DFT ise sonlu uzunluklu bir dizinin yalnızca belirli frekans noktalarındaki örneklerini verir. FFT'ye uygulanan pencereleme sonrasında elde edilen dizi $x_N[n]$ için DFT

$$X[k] = \sum_{n=0}^{N-1} x_N[n]e^{-j2\pi kn/N}$$

şeklindedir. Burada

$$k = 0, 1, 2, \dots, N-1 \quad (14)$$

olup DFT frekans noktaları

$$\omega_k = \frac{2\pi k}{N}$$

olarak tanımlanır. Dolayısıyla DFT, pencere uygulanmış sonlu uzunluklu ayırık zaman sinyalinin DTFT'sinin örneklenmiş halidir:

$$X[k] = X_N(e^{j\omega}) \Big|_{\omega=2\pi k/N} \quad (15)$$

Bu projede kullanılan FFT IP, işte bu DFT'nin donanımda hızlı hesaplanan algoritmik karşılığıdır.

2.3 Rectangular Window in Frequency Domain

Zaman alanında pencereleme yapılması frekans alanında konvolüsyona karşılık gelir:

$$x_N[n] = x[n]w_R[n] \quad (16)$$

$$X_N(e^{j\omega}) = \frac{1}{2\pi} X(e^{j\omega}) * W_R(e^{j\omega}) \quad (17)$$

Burada $W_R(e^{j\omega})$ rectangular window'un DTFT'sidir:

$$W_R(e^{j\omega}) = \sum_{n=0}^{N-1} e^{-j\omega n}$$

Geometrik seri açılımı kullanıldığında

$$W_R(e^{j\omega}) = e^{-j\omega(N-1)/2} \frac{\sin(N\omega/2)}{\sin(\omega/2)}$$

elde edilir. Büyüklük cevabı ise

$$|W_R(e^{j\omega})| = \left| \frac{\sin(N\omega/2)}{\sin(\omega/2)} \right|$$

şeklindedir. Bu ifade sinc/Dirichlet kernel benzeri ana lob ve yan loblardan oluşan bir spektral yapıyı temsil eder. Dolayısıyla rectangular window zaman alanında dikdörtgen olsa da, frekans alanında sinc-benzeri bir yayılmaya neden olur. Bu yüzden doğru yorum şudur: zaman alanında rectangular window ile çarpma, frekans alanında rectangular window'un spektrumu ile konvolüsyona karşılık gelir.

2.4 Origin of Spectral Leakage

İdeal sonsuz uzunluklu kompleks üstel sinyal

$$x[n] = Ae^{j\omega_0 n} \quad (18)$$

olduğunda DTFT alanında tek frekansta impulse-benzeri bir yapı verir:

$$X(e^{j\omega}) = 2\pi A \sum_{m=-\infty}^{\infty} \delta(\omega - \omega_0 - 2\pi m) \quad (19)$$

Ancak pratikte bu sonsuz sinyalin yalnızca 256 örnek uzunluğundaki kısmı alınır:

$$x_N[n] = x[n]w_R[n] \quad (20)$$

Bu nedenle frekans alanında impulse doğrudan gözlenmez; rectangular window'un spektrumu ile yayılmış bir yapı oluşur. Bu yayılma spectral leakage'in temel nedenidir. Bir başka deyişle, ideal olarak tek frekansta bulunan enerji, rectangular window'un ana lobu ve yan lobları boyunca çevre frekanslara da dağılır.

2.5 FFT Resolution and Expected Bin Locations

FFT frekans çözünürlüğü

$$\Delta f = \frac{f_s}{N} \quad (21)$$

ile verilir. Bu projede

$$f_s = 200 \text{ MHz}, \quad N = 256 \quad (22)$$

olduğundan

$$\Delta f = \frac{200 \text{ MHz}}{256} = 781.25 \text{ kHz} \quad (23)$$

elde edilir. Bir giriş frekansının teorik FFT bin değeri

$$k_0 = \frac{f_0}{\Delta f} = \frac{f_0 N}{f_s} \quad (24)$$

şeklinde hesaplanır.

12.5 MHz için

$$k_0 = \frac{12.5 \text{ MHz}}{781.25 \text{ kHz}} = 16 \quad (25)$$

olur. Bu değer tam sayı olduğu için sinyal FFT bin merkezine tam oturur. Bu durumda rectangular window spektrumunun sıfır noktaları DFT'nin örnekleme noktaları ile hizalanır ve ideal durumda enerji yalnızca 16. bin üzerinde görülür.

10 MHz için ise

$$k_0 = \frac{10 \text{ MHz}}{781.25 \text{ kHz}} = 12.8 \quad (26)$$

elde edilir. Bu değer tam sayı değildir. Dolayısıyla sinyal hiçbir FFT bin'ine tam olarak oturmaz. Bu durumda DFT, rectangular window spektrumunun ana lob ve yan lob bölgelerinden örnekler alır. Sonuç olarak enerji tek bir bin'de kalmaz; komşu bin'lere yayılır. 10 MHz sinyali 13. bin'e daha yakın olduğu için peak değerinin yaklaşık olarak 13 civarında oluşması beklenir.

2.6 Magnitude-Squared Peak Detection

FFT IP çıkışı kompleks olduğundan her bin için

$$X[k] = X_{\text{real}}[k] + jX_{\text{imag}}[k] \quad (27)$$

şeklinde bir ifade söz konusudur. Spektral peak'i bulmak için büyüklük karesi hesaplanır:

$$|X[k]|^2 = X_{\text{real}}^2[k] + X_{\text{imag}}^2[k]$$

Donanım açısından karekök işlemi maliyetli olduğu ve maksimum bin konumunu değiştirmedeği için doğrudan $|X[k]|^2$ kullanılmıştır. Peak bin şu şekilde tanımlanır:

$$k_{\text{peak}} = \arg \max_k (|X[k]|^2)$$

Bu matematiksel ifade donanım tarafında `abs_sq.v` ve `peak_detector.v` modülleri ile gerçekleştirilmiştir. `abs_sq.v` her FFT çıkışı için real ve imaginary bileşenlerin kareleri toplamını üretirken, `peak_detector.v` bir FFT frame'i boyunca en büyük $|X[k]|^2$ değerini ve buna karşılık gelen `xk_index` değerini saklamaktadır.

2.7 Interpretation for This Project

Sonuç olarak Part 1'de yapılan işlem, sürekli zamanlı bir sinyalin doğrudan Fourier dönüşümünü almak değildir. Python ve FPGA tarafında işlenen sinyal, sürekli zamanlı kompleks üstelin $f_s = 200 \text{ MHz}$ ile örneklenmiş ayrık zaman karşılığıdır. Bu ayrık zaman dizisinin yalnızca 256 örneği FFT'ye verildiği için gerçekleştirilen işlem rectangular window uygulanmış bir DFT olarak yorumlanmalıdır. Bu çerçevede, 12.5 MHz sinyalde tam bin hizalanması nedeniyle temiz bir peak oluşmasını; 10 MHz sinyalde ise tam bin hizalanması olmadığı için leakage gözlenmesini açıklamaktadır. Sonraki bölümlerde de aynı terminoloji izlenecektir: `tb_fft_naive.v` Python'daki $x[n]$ dizisini üretir

veya okur, `fft_naive_top.v` bu $x_N[n]$ frame'ini FFT IP'ye AXI-Stream ile verir, `abs_sq.v` $|X[k]|^2$ hesabını yapar ve `peak_detector.v` ise $\arg \max$ işlemini donanımsal olarak gerçekleştirir.

3 Python Reference Model

Part 1 için kullanılan Python referans dosyası `reference_fft.py`'dir. Bu dosya, FPGA doğrulama akışının yazılım tarafındaki referansını üretmekte ve testbench'e beslenen giriş örneklerini hazırlamaktadır.

Dosyanın görevleri aşağıdaki gibidir:

1. 12.5 MHz kompleks sinyal üretmek,
2. 10 MHz kompleks sinyal üretmek,
3. real ve imag bileşenleri 12-bit signed fixed-point formata çevirmek,
4. testbench tarafından okunacak `.mem` dosyalarını üretmek,
5. Python FFT ile beklenen tepe frekans bileşeni değerlerini kontrol etmek.

Üretilen bellek dosyaları:

- `tone_12p5MHz_re.mem`
- `tone_12p5MHz_im.mem`
- `tone_10MHz_re.mem`
- `tone_10MHz_im.mem`

Temel parametreler Tablo 1'de verilmiştir.

Parametre	Değer
FS	200e6
N	256
INPUT_BITS	12
AMP	0.9

Tablo 1: Python referans modelinde kullanılan temel parametreler

Kullanılan temel fonksiyonlar şunlardır:

- `generate_complex_tone(freq_hz)`: Kompleks üstel üretir.

$$x[n] = Ae^{j2\pi f_0 n/f_s} \quad (28)$$

- `quantize_signed(x)`: Floating-point sinyali 12-bit signed integer formata dönüştürür.
- `to_twos_hex(value, bits)`: Signed integer değeri, `.mem` dosyasına yazılacak hexadecimal two's complement formata çevirir.
- `write_complex_mem(freq_hz, name)`: Real ve imag bileşenler için ayrı bellek dosyaları üretir.
- `fft_report(freq_hz)`: Python FFT ile teorik frekans bileşeni ve tepe frekans bileşeni kontrolü yapar.

Ayrıca Part 2 için ayrı bir Python dosyası bulunmaktadır: `generate_hamming_mem.py`. Bu dosya Hamming window katsayılarını üretir ve `hamming_q1_15.mem` dosyasını oluşturur. Ancak bu dosya Part 1'de kullanılan naive FFT akışına dahil edilmemiştir.

4 Xilinx FFT IP Configuration

Part 1 tasarımında kullanılan Xilinx FFT IP ayarları Tablo 2 içerisinde verilmiştir.

Parametre	Değer
FFT IP	<code>xfft_0</code>
Transform length	256
Clock	200 MHz
Input data width	12 bit
Output data width	12 bit
Data format	Fixed-point
Architecture	Pipelined Streaming I/O
Output ordering	Natural order
<code>xk_index</code>	Enabled
AXI input <code>tdata</code> width	32 bit
AXI output <code>tdata</code> width	32 bit
AXI <code>tuser</code> width	8 bit
FFT_CONFIG_TDATA	<code>16'h0155</code>

Tablo 2: Xilinx FFT IP yapılandırma parametreleri

FFT IP, 12-bit real ve 12-bit imag veri ile çalışacak şekilde seçilmiştir. Bununla birlikte AXI-Stream `tdata` portunda bu iki alan doğrudan 24 bit olarak değil, 16-bit lane hizalaması ile taşınmaktadır. Bu nedenle toplam taşıma genişliği 32 bittir.

$$\text{AXIS_DATA_WIDTH} = 16_{\text{real lane}} + 16_{\text{imag lane}} = 32 \quad (29)$$

Bu projede $\text{DATA_WIDTH} = 12$, gerçek FFT veri çözünürlüğünü ifade eder. $\text{AXIS_LANE_WIDTH} = 16$ ise Xilinx FFT IP'nin AXI-Stream `tdata` portunda 12-bit real ve imag değerleri 16-bit lane'lere hizalamasından kaynaklanır. Bu nedenle $\text{AXIS_DATA_WIDTH} = 32$ olarak kullanılmıştır.

12-bit signed değerler 16-bit lane'e sign-extension ile yerleştirilmiştir. Pozitif sayılarda üst bitler 0 ile, negatif sayılarda ise 1 ile doldurulur. Böylece two's complement işaret bilgisi korunur.

Örnek olarak 12-bit -1 değeri `1111_1111_1111`, 16-bit -1 değeri ise `1111_1111_1111_1111` biçimindedir.

Kullanılan paketleme mantığı aşağıdaki gibidir:

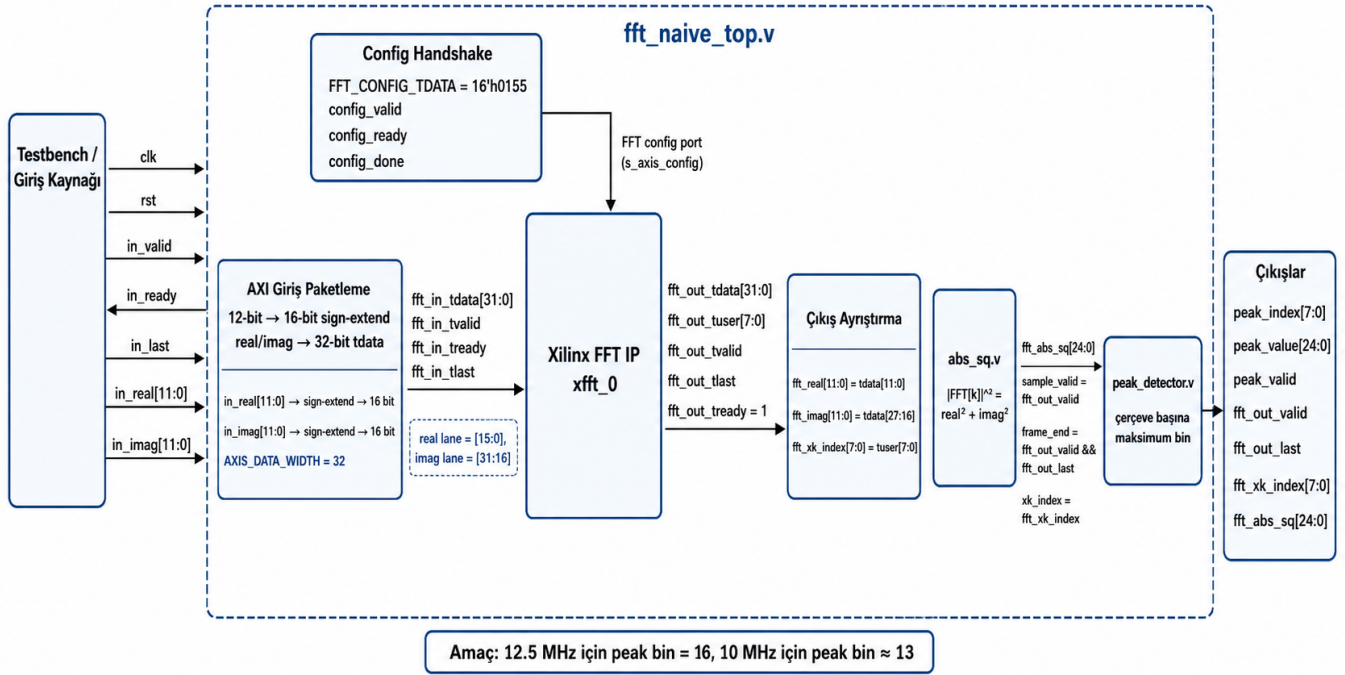
```
assign fft_in_tdata = {
    {(AXIS_LANE_WIDTH-DATA_WIDTH){in_imag[DATA_WIDTH-1]}},
    in_imag,
    {(AXIS_LANE_WIDTH-DATA_WIDTH){in_real[DATA_WIDTH-1]}},
    in_real
};
```

Burada $\text{AXIS_LANE_WIDTH} - \text{DATA_WIDTH} = 16 - 12 = 4$ olduğundan, her 12-bit signed değer başına 4 adet işaret biti eklenmiştir.

5 Hardware Architecture and Data Flow

Part 1 veri yolu, testbench tarafından üretilen kompleks örneklerin AXI-Stream arayüzü üzerinden FFT IP'ye beslenmesi ve çıkışta güç spektrumunun hesaplanmasına dayanmaktadır.

Part 1 Veri Akışı Blok Diyagramı



Şekil 1: Part 1 Naive FFT veri akışı blok diyagramı

Şekil 1’de Part 1 için kurulan veri yolu gösterilmiştir. Testbench tarafından üretilen 12-bit signed real/imag girişler önce AXI-Stream veri formatına paketlenir. Xilinx FFT IP’ye config handshake tamamlandıktan sonra veri aktarılır. FFT IP çıkışında real ve imag bileşenleri ayrıştırılır, `abs_sq.v` modülü ile $|X[k]|^2 = \text{real}^2 + \text{imag}^2$ hesabı yapılır. `peak_detector.v` modülü her 256 örneklilik FFT frame’i boyunca maksimum `abs_sq` değerini takip ederek frame sonunda `peak_index` ve `peak_value` sonuçlarını üretir.

Veri akışı metin olarak aşağıdaki sırayı izlemektedir:

Testbench / giriş kaynağı → AXI giriş paketleme → Xilinx FFT IP xfft_0 → FFT çıkış ayrıştırma → `abs_sq.v` → `peak_detector.v` → `peak_index`, `peak_value`, `peak_valid`

6 Verilog Module Descriptions

6.1 fft_naive_top.v

Top modül, Part 1 tasarımının kontrol ve entegrasyon katmanını oluşturmaktadır. Bu modülün temel görevleri şunlardır:

1. FFT IP config kanalını yönetmek,

2. config tamamlanmadan input data gönderilmesini engellemek,
3. 12-bit real/imag veriyi 16-bit lane'lere sign-extend ederek 32-bit AXI tdata formatına paketlemek,
4. FFT IP çıkışından real, imag ve `xk_index` sinyallerini ayırmak,
5. `abs_sq` ve `peak_detector` modüllerini bağlamak,
6. testbench veya üst modül için debug ve sonuç sinyallerini dışarı vermek.

Config handshake tarafında `config_valid`, `config_ready` ve `config_done` sinyalleri kullanılmaktadır. Config kabul edilene kadar giriş stream'i başlatılmaz. Kullanılan mantık aşağıdaki gibidir:

```
assign fft_in_tvalid = in_valid & config_done;
assign in_ready      = fft_in_ready & config_done;
```

Bu yapı, FFT IP konfigürasyonu tamamlanmadan ilk input frame'in gönderilmesini engeller. Böylece ilk frame'in yanlış FFT config ile işlenmesi önlenir.

FFT çıkış ayrıştırma mantığı aşağıdaki şekilde özetlenebilir:

```
fft_real      = fft_out_tdata[11:0]
fft_imag      = fft_out_tdata[27:16]
fft_xk_index  = fft_out_tuser[7:0]
```

Xilinx FFT IP'nin real bileşeni `tdata`'nın alt lane'inde, imag bileşeni ise üst lane'inde taşınmaktadır. Her iki bileşen signed 12-bit olarak yorumlanmaktadır.

6.2 `abs_sq.v`

`abs_sq.v` modülü, FFT çıktısının her frekans bileşeni için güç büyüklüğünü hesaplamaktadır. Matematiksel olarak gerçekleştirilen işlem aşağıdaki gibidir:

$$|X[k]|^2 = \text{Re}\{X[k]\}^2 + \text{Im}\{X[k]\}^2 \quad (30)$$

Verilog düzeyinde bu işlem,

```
real_sq = real_in * real_in
imag_sq = imag_in * imag_in
abs_sq  = real_sq + imag_sq
```

şeklinde gerçekleştirilmiştir.

Bit genişliği analizi açısından FFT çıkışının 12-bit signed olması nedeniyle yaklaşık giriş aralığı

$$-2048 \leq \text{real}, \text{imag} \leq 2047 \quad (31)$$

olarak düşünülebilir. Maksimum teorik güç ise,

$$|X[k]|_{\max}^2 \approx 2048^2 + 2048^2 = 8,388,608 \quad (32)$$

değerine ulaşır. 25-bit unsigned üst sınır ise,

$$2^{25} = 33,554,432 \quad (33)$$

olduğundan `abs_sq[24:0]` genişliği bu tasarım için yeterlidir. Simülasyon sonuçlarında gözlenen maksimum değerler 3.4 milyon civarında olduğundan overflow gözlenmemiştir.

6.3 `peak_detector.v`

`peak_detector.v` modülü, her FFT frame'i boyunca oluşan maksimum güç değerini ve buna karşılık gelen frekans bileşeni indeksini takip eder. Modül frame bazlı çalışmaktadır.

Girişler:

- `sample_valid = fft_out_valid`
- `frame_end = fft_out_valid && fft_out_last`
- `abs_sq`: o andaki FFT frekans bileşeni gücü
- `xk_index`: o andaki FFT frekans bileşeni indeksi

Çıkışlar:

- `peak_index`: frame boyunca görülen maksimum gücün indeksi
- `peak_value`: frame boyunca görülen maksimum `abs_sq` değeri
- `peak_valid`: frame sonunda 1 clock pulse

Bu modülün zamanlama yorumu kritik öneme sahiptir: `fft_xk_index` ve `fft_abs_sq` anlık FFT çıkış frekans bileşenini gösterir. `peak_index` ve `peak_value` ise bir frame boyunca taranan maksimum değer frame sonunda register'a alınmış halidir. Bu nedenle `peak_index = 13` olduğu anda `fft_xk_index`'in de 13 olması beklenmez. 10 MHz frame sırasında `fft_xk_index = 13` olduğunda `fft_abs_sq` maksimuma ulaşır; frame tamamlandığında ise `peak_detector` bu sonucu `peak_index = 13` ve `peak_value = 2968146` olarak raporlar.

Ek olarak şu AXI-Stream valid uyarısı geçerlidir: `fft_out_valid = 0` iken `fft_xk_index` ve `fft_abs_sq` bus üzerinde eski veya stale değerler tutabilir. Bu değerler geçerli FFT sonucu değildir. Waveform yorumları yalnızca `fft_out_valid = 1` iken yapılmalıdır.

7 SystemVerilog Testbench

`tb_fft_naive.sv` testbench'i yalnızca giriş uygulayan basit bir sürücü değil, aynı zamanda donanım davranışını ölçen ve sonuçları kontrol eden kapsamlı bir doğrulama ortamıdır. Temel görevleri aşağıda verilmiştir:

1. 200 MHz clock üretmek,
2. reset uygulamak,
3. Python tarafından üretilen `.mem` dosyalarını okumak,
4. 12.5 MHz ve 10 MHz giriş frame'lerini ardışık olarak göndermek,
5. AXI valid/ready handshake'e uygun veri aktarımı yapmak,
6. FFT output valid olduğunda `xk_index` ve `abs_sq` değerlerini izlemek,
7. `peak_valid` geldiğinde `peak_index` ve `peak_value` sonuçlarını kaydetmek,
8. frame energy hesabı yapmak,
9. peak dominance hesabı yapmak,
10. beklenen tepe frekans bileşeni değerlerine göre assertion/FAIL kontrolü yürütmek.

Saat periyodu ve frekansı aşağıdaki gibidir:

$$T_{clk} = 5 \text{ ns} \quad (34)$$

$$f_{clk} = \frac{1}{T_{clk}} = 200 \text{ MHz} \quad (35)$$

Giriş frame'leri aşağıdaki çağrılar ile uygulanmaktadır:

```
send_frame("12.5 MHz", tone12_re, tone12_im);  
send_frame("10.0 MHz", tone10_re, tone10_im);
```

Bellek dosyaları ise şu biçimde okunmaktadır:

```

$readmemh("tone_12p5MHz_re.mem", tone12_re);
$readmemh("tone_12p5MHz_im.mem", tone12_im);
$readmemh("tone_10MHz_re.mem", tone10_re);
$readmemh("tone_10MHz_im.mem", tone10_im);

```

`send_frame` task'i her örneği yalnızca `in_ready` aktif olduğunda gönderir. Bu sayede FFT IP backpressure uygularsa input data kaybı yaşanmaz.

Toplam frame enerjisi aşağıdaki gibi tanımlanır:

$$E_{\text{total}} = \sum_{k=0}^{N-1} |X[k]|^2 \quad (36)$$

Ortalama frekans bileşeni enerjisi ise:

$$E_{\text{mean}} = \frac{E_{\text{total}}}{N} \quad (37)$$

Peak dominance metriği de:

$$D = \frac{\max_k |X[k]|^2}{E_{\text{mean}}} \quad (38)$$

İdeal olarak FFT frekans bileşeni merkezine tam oturan tek frekanslı kompleks üstel için,

$$X[k] = 0, \quad k \neq k_0 \quad (39)$$

$$E_{\text{total}} \approx |X[k_0]|^2 \quad (40)$$

$$D = \frac{|X[k_0]|^2}{|X[k_0]|^2 / N} = N \quad (41)$$

elde edilir. Bu projede $N = 256$ olduğundan ideal durumda $D \approx 256$ beklenir. 12.5 MHz sinyali FFT frekans bileşeni merkezine tam oturduğundan ideal durumda dominance oranının yaklaşık 256 olması beklenir. 10 MHz sinyalinde leakage nedeniyle enerji birden fazla frekans bileşenine yayıldığından dominance oranı 256'dan daha küçük olur. Ancak sinyal hâlâ baskın bir tek frekans bileşeni içerdiği için peak gücünün ortalama frekans bileşeni gücünden belirgin biçimde büyük kalması beklenir.

Testbench'te `PEAK_DOMINANCE_MIN = 4` seçilmiştir. Bu değer teorik ideal değer değildir. Bu eşik, spektrumun tamamen bozulmadığını ve belirgin bir peak bulunduğunu doğrulayan gevşek fakat anlamlı bir güvenlik kontrolüdür. Ana doğrulama tepe frekans bileşeni konumlarıyla yapılmaktadır.

8 Simulation Results

Simülasyon logları aşağıda verilmiştir:

```
[158000] === Frame 0: Sending frame: 12.5 MHz ===
[1184000] === Frame 1: Sending frame: 10.0 MHz ===
[2662000] [Frame0 12.5MHz] xk=10 abs_sq=0
[2666000] [Frame0 12.5MHz] xk=11 abs_sq=0
[2670000] [Frame0 12.5MHz] xk=12 abs_sq=0
[2674000] [Frame0 12.5MHz] xk=13 abs_sq=0
[2678000] [Frame0 12.5MHz] xk=14 abs_sq=0
[2682000] [Frame0 12.5MHz] xk=15 abs_sq=0
[2686000] [Frame0 12.5MHz] xk=16 abs_sq=3392965
[2690000] [Frame0 12.5MHz] xk=17 abs_sq=0
[2694000] [Frame0 12.5MHz] xk=18 abs_sq=0
[3642000] frame 0 total energy = 3392966 (bins=256)
[3646000] peak[0]: index=16 value=3392965
[3686000] [Frame1 10.0MHz] xk=10 abs_sq=15426
[3690000] [Frame1 10.0MHz] xk=11 abs_sq=36941
[3694000] [Frame1 10.0MHz] xk=12 abs_sq=185434
[3698000] [Frame1 10.0MHz] xk=13 abs_sq=2968146
[3702000] [Frame1 10.0MHz] xk=14 abs_sq=82645
[3706000] [Frame1 10.0MHz] xk=15 abs_sq=24484
[3710000] [Frame1 10.0MHz] xk=16 abs_sq=11642
[3714000] [Frame1 10.0MHz] xk=17 abs_sq=6740
[3718000] [Frame1 10.0MHz] xk=18 abs_sq=4394
[4666000] frame 1 total energy = 3391331 (bins=256)
[4670000] peak[1]: index=13 value=2968146
-----
Frame 0 (12.5 MHz): tepe frekans bileşeni=16, |X|^2=3392965, total=3392966
Frame 1 (10.0 MHz): tepe frekans bileşeni=13, |X|^2=2968146, total=3391331
Energy ratio (smaller/larger) = 1.000 (tol >= 0.750)
Peak dominance: tone0=256.0x, tone1=224.1x (min=4)
-----
tb_fft_naive: PASS
```

Bu loglar iki temel DSP gözlemini doğrudan doğrulamaktadır. İlk frame'de 12.5 MHz giriş için `fft_out_valid = 1` iken `fft_xk_index = 16` ve `fft_abs_sq = 3392965` birlikte gözlenmiştir. Bu davranış, `fft_abs_sq` sinyalinin anlık FFT çıkış frekans bileşeni ile eşzamanlı güncellendiğini göstermektedir. İlgili frekans bileşeni dışında anlamlı güç görülmemesi, 12.5 MHz tonunun spektrumda tek ve dar bir çizgi olarak ortaya çıktığını göstermektedir. Buna karşılık 10 MHz frame'de en büyük anlık değer `fft_xk_index = 13` üzerinde `fft_abs_sq = 2968146` olarak oluşmuştur; ancak `xk_index = 12, 14, 15` gibi komşu frekans bileşenlerinde de sıfır olmayan değerler gözlenmektedir. Bu durum spectral leakage etkisinin simülasyonda açık biçimde gözlendiğini doğrulamaktadır.

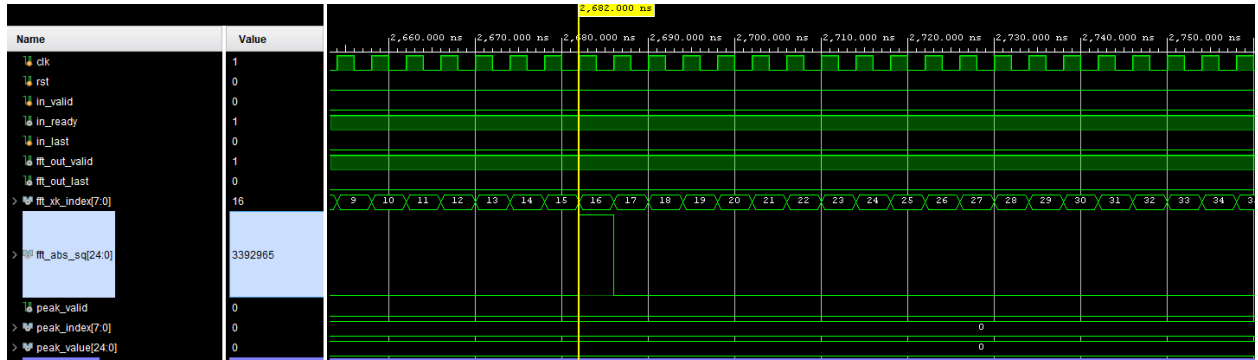
Frame	Frequency	Theoretical frequency component	Measured peak component	Peak value	Total energy	Comment
0	12.5 MHz	16.0	16	3392965	3392966	Component-centered, minimum leakage
1	10.0 MHz	12.8	13	2968146	3391331	Off-component-centered, leakage observed

Tablo 3: Simülasyon sonuçlarının özet tablosu

Sonuçlar ayrıca enerji tutarlılığı açısından da anlamlıdır. İki frame'in toplam enerjisinin birbirine çok yakın olması, giriş genliğinin tutarlı olduğunu ve gözlenen farkın genlik değişiminden değil, frekans bileşeni yerleşiminden kaynaklandığını göstermektedir. Dominance oranlarının sırasıyla 256.0x ve 224.1x olması da bu yorumu desteklemektedir.

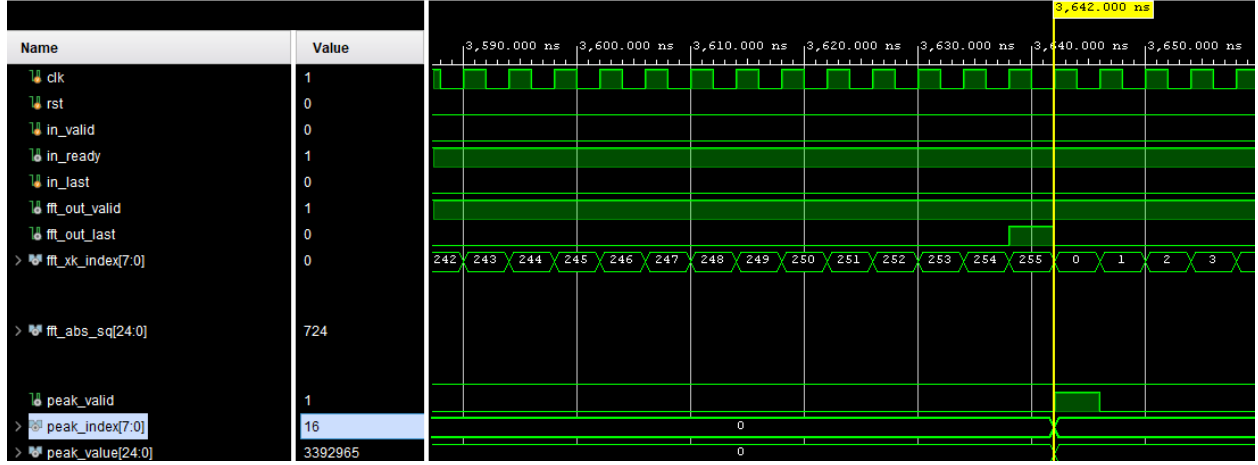
9 Waveform Analysis

Bu bölümde kullanıcı tarafından eklenen dört waveform görüntüsü doğrudan rapora yerleştirilmiştir. Görseller birlikte incelendiğinde iki kritik zamanlama davranışı açıkça görülmektedir: `fft_abs_sq` yalnızca `fft_out_valid` aktifken anlık `fft_xk_index` ile birlikte güncellenmektedir; buna karşılık `peak_index` ve `peak_value` sinyalleri frame sonunu gösteren `fft_out_last` olayını beklemekte ve `peak_valid` darbesi ile geçerli hale gelmektedir.



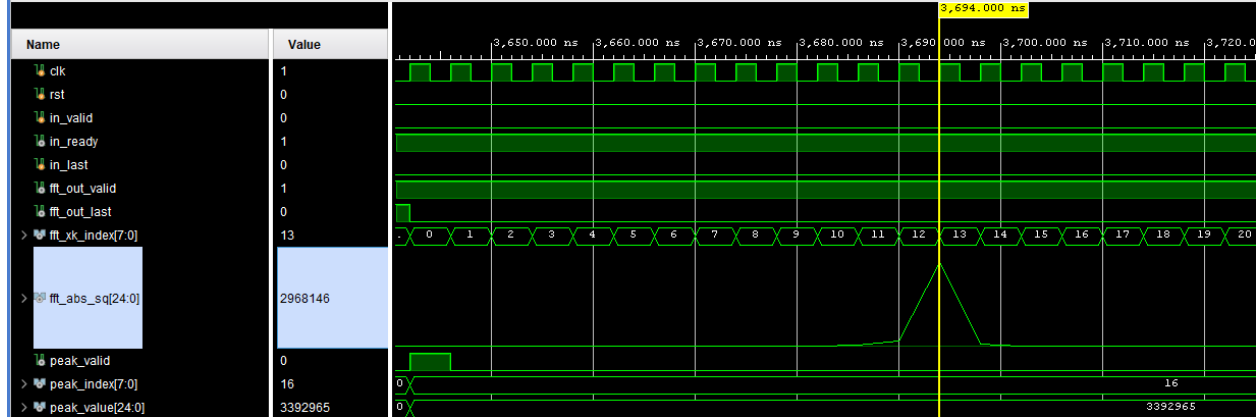
Şekil 2: 12.5 MHz için anlık FFT çıkış frekans bileşeni ve `fft_abs_sq` güncellemesi

Şekil 2'de `fft_out_valid` = 1 iken `fft_xk_index` = 16 ve `fft_abs_sq` = 3392965 birlikte görülmektedir. Bu görünüm, `fft_abs_sq` değerinin anlık FFT çıkış frekans bileşeni ile aynı çevrimde güncellendiğini göstermektedir. 12.5 MHz sinyalinin enerjisi pratik olarak tek bir frekans bileşeninde toplandığı için spektrumda yüksek ve ince tek bir çizgi oluşmaktadır.



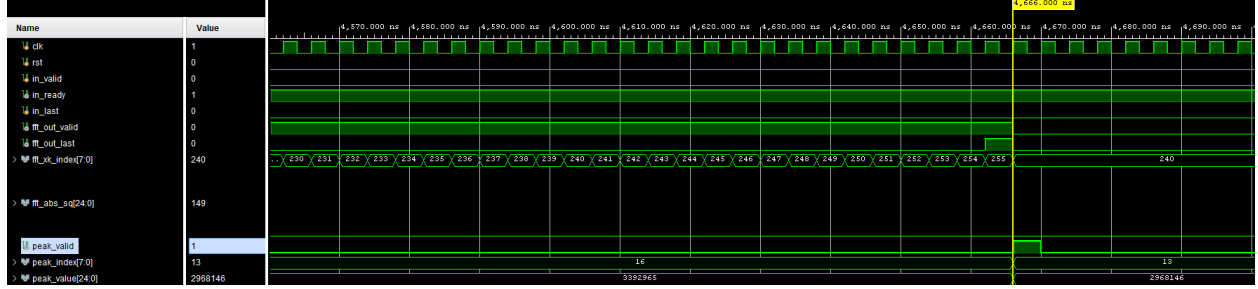
Şekil 3: 12.5 MHz frame sonunda peak_index ve peak_value üretimi

Şekil 3’de frame sonuna yakın bölgede `fft_out_last` olayından hemen sonra `peak_valid` darbesi oluşmakta, ardından `peak_index = 16` ve `peak_value = 3392965` register’lara alınmaktadır. Bu durum, anlık `fft_abs_sq` ile frame sonu özeti olan `peak_index/peak_value` sinyalleri arasındaki farkı açık biçimde göstermektedir.



Şekil 4: 10 MHz için leakage bölgesi ve komşu frekans bileşeni enerjileri

Şekil 4’de cursor `fft_xk_index = 13` anına yerleştirilmiştir ve bu anda `fft_abs_sq = 2968146` ölçülmektedir. Bununla birlikte güç yalnızca 13. frekans bileşeninde toplanmamış; komşu frekans bileşenlerine doğru analog görünümlü bir güç dağılımı oluşmuştur. Bu davranış, 10 MHz tonunun FFT frekans bileşeni merkezine tam oturmaması nedeniyle oluşan spectral leakage etkisini doğrulamaktadır.



Şekil 5: 10 MHz frame sonunda tepe frekans bileşeni raporlaması

Şekil 5'te ikinci frame tamamlandıktan sonra **peak_valid** darbesi oluşmakta ve **peak_index** = 13, **peak_value** = 2968146 değerleri raporlanmaktadır. Bu anda **fft_xk_index** hattı artık anlık akış bilgisini taşımakta veya geçersiz kalabilmektedir; dolayısıyla frame sonu özeti ile aynı değerde olması beklenmez.

Bu dört görsel birlikte değerlendirildiğinde şu sonuç ortaya çıkmaktadır: **fft_abs_sq**, yalnızca **fft_out_valid** aktifken anlamlı olan anlık FFT çıkış gücünü temsil eder; **peak_index** ve **peak_value** ise tüm frame tarandıktan sonra **fft_out_last** ve **peak_valid** ile anlam kazanan özet sonuçlardır.

10 Timing and AXI-Stream Interpretation

AXI-Stream arayüzünde **tdata** ve **tuser** sinyalleri yalnızca **tvalid** aktifken geçerli kabul edilir. FFT IP çıkışında **fft_out_valid** = 0 olduğunda **fft_xk_index** ve **fft_abs_sq** bus üzerinde eski değerleri tutabilir. Simülasyon sırasında **fft_xk_index**'in 240 gibi bir değerde kalması bu nedenle hata olarak değerlendirilmemiştir.

Doğru yorumlama kuralları aşağıdaki gibidir:

- **fft_out_valid** = 1 iken **fft_xk_index** anlamlıdır.
- **fft_out_valid** = 1 iken **fft_abs_sq** anlamlıdır.
- **peak_valid** = 1 iken **peak_index** anlamlıdır.
- **peak_valid** = 1 iken **peak_value** anlamlıdır.

Bu iki olayın aynı anda gerçekleşmesi gerekmez. Birincisi akış halindeki anlık FFT çıkışını, ikincisi ise frame sonunda özetlenmiş maksimum sonucu temsil etmektedir.

11 Part 1 Conclusion

Part 1 kapsamında Xilinx FFT IP kullanılarak 256 noktalı naive FFT veri yolu başarıyla kurulmuştur. Python referans modeli ile üretilen 12-bit signed kompleks giriş örnekleri, SystemVerilog

testbench tarafından okunmuş ve AXI-Stream handshake kurallarına uygun şekilde FFT IP'ye aktarılmıştır. FFT çıkışında real ve imag bileşenleri ayrıştırılmış, Verilog ile $|X[k]|^2$ hesabı yapılmış ve `peak_detector` modülü ile her frame için maksimum güç frekans bileşeni bulunmuştur.

12.5 MHz sinyali için teorik frekans bileşeni değeri 16 olup, simülasyonda `peak_index = 16` ve `peak_value = 3392965` ölçülmüştür. 10 MHz sinyali için teorik frekans bileşeni değeri 12.8 olup, simülasyonda maksimum güç `xk_index = 13` üzerinde oluşmuş ve frame sonunda `peak_index = 13`, `peak_value = 2968146` olarak raporlanmıştır. 10 MHz frame'de komşu frekans bileşenlerinde de sıfır olmayan güç değerlerinin görülmesi leakage etkisini doğrulamaktadır.

Testbench ayrıca toplam spektral enerji ve peak dominance kontrolleri yapmıştır. Bu bölümde kullanılan toplam enerji, ortalama frekans bileşeni enerjisi ve dominance tanımları aşağıdaki gibidir:

$$E_{\text{total}} = \sum_{k=0}^{N-1} |X[k]|^2$$

$$E_{\text{mean}} = \frac{E_{\text{total}}}{N}$$

$$D = \frac{\max_k |X[k]|^2}{E_{\text{mean}}}$$

İki frame'in toplam enerjilerinin birbirine çok yakın olması input genliklerinin tutarlı olduğunu göstermektedir. 12.5 MHz için dominance oranı yaklaşık 256.0x, 10 MHz için ise 224.1x olarak ölçülmüştür. Bu sonuçlar, FFT frekans bileşeni merkezine tam oturan sinyalde enerjinin tek frekans bileşeninde yoğunlaştığını, merkeze tam oturmeyen sinyalde ise enerjinin komşu frekans bileşenlerine yayıldığını göstermektedir.

Sonuç olarak Part 1 tasarımı, Python referans modeli, Verilog implementasyonu ve SystemVerilog testbench doğrulaması ile beklenen FFT ve leakage davranışını başarıyla göstermiştir. Bu çalışma, hem FPGA veri yolu entegrasyonu hem de DSP davranışının fixed-point ve AXI-Stream bağlamında doğrulanması açısından sağlam bir temel ortaya koymaktadır.

12 Part 2: Time-Domain Hamming Window Implementation on FPGA

Bu bölüm, Part 1’de doğrulanan naive FFT zincirinin zaman alanı pencereleme ile genişletilmiş sürümünü sunmaktadır. Part 1’de 12.5 MHz ve 10 MHz kompleks üstel girişler doğrudan FFT IP’ye uygulanmış, rectangular window etkisi altında özellikle 10 MHz tone için spectral leakage açık biçimde gözlenmişti.

Part 2’de amaç, aynı FFT akışını koruyarak FFT girişinden önce Hamming window uygulamaktır. Böylece bin merkezine tam oturmayan sinyallerde uzak yan lobların azaltılması ve enerjinin daha kontrollü bir biçimde ana lob çevresinde toplanması hedeflenmiştir.

Part 2, Part 1’de kurulan naive FFT veri yolunu genişletmektedir. FFT IP, AXI-Stream arayüzü, abs_sq hesabı ve peak_detector yapısı korunmuş; giriş veri yoluna ek olarak zaman alanı Hamming pencereleme bloğu eklenmiştir. Python tarafında üretilen katsayılar fixed-point formata çevrilmiş, hamming_q1_15.mem dosyasına yazılmış ve Verilog tarafında \$readmemh ile okunarak her örneğe sample index bazında uygulanmıştır.

13 Theoretical Background for Part 2

Part 1’de tartışıldığı gibi, naive FFT akışı pratikte rectangular window uygulanmış bir DFT üretmektedir. Part 2’de değişen tek nokta, aynı çerçevenin Hamming window ile yeniden ağırlıklandırılmasıdır. Bu nedenle odak artık FFT altyapısında değil, rectangular window’dan Hamming window’a geçildiğinde leakage dağılımının nasıl değiştiğindedir.

Part 1’de FFT’ye verilen sonlu uzunluklu sinyal

$$x_N[n] = x[n]w_R[n] \quad (42)$$

şeklinde yorumlanmıştı. Burada rectangular window

$$w_R[n] = \begin{cases} 1, & 0 \leq n \leq N-1 \\ 0, & \text{diğer durumlarda} \end{cases} \quad (43)$$

ifadesi ile tanımlanır. Rectangular window’un frekans cevabı dar bir ana lob ve yüksek yan loblar içerdiğinden, sinyal FFT bin merkezine tam oturmadığında enerji komşu bin’lere ve daha uzak frekans bileşenlerine yayılır.

Part 2’de rectangular window yerine Hamming window kullanılmaktadır. Kullanılan Python modeliyle uyumlu olacak şekilde pencere katsayısı

$$w_H[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N}\right), \quad 0 \leq n \leq N-1$$

olarak alınmıştır. Bu projede $N = 256$ olduğundan

$$w_H[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{256}\right) \quad (44)$$

elde edilir. Hamming window uygulanmış giriş dizisi

$$x_w[n] = x[n]w_H[n]$$

şeklindedir ve FFT'ye artık doğrudan $x[n]$ değil, $x_w[n]$ uygulanır. Zaman alanındaki bu çarpım frekans alanında konvolüsyona karşılık gelir:

$$X_w(e^{j\omega}) = \frac{1}{2\pi} X(e^{j\omega}) * W_H(e^{j\omega}) \quad (45)$$

Hamming window, rectangular window'a göre yan lobları bastırır. Bu nedenle bin merkezine tam oturmeyen sinyallerde uzak komşu binlere yayılan leakage azalır. Bunun bedeli ana lobun bir miktar genişlemesi ve peak genliğinin pencere kazancı nedeniyle azalmasıdır. Hamming window leakage'i tamamen yok etmez; enerjiyi daha düşük yan loblarla daha kontrollü bir dağılıma dönüştürür. Bu nedenle karşılaştırma yalnızca peak değerine göre değil, komşu binlerdeki enerji dağılımına göre yapılmalıdır.

14 Python Hamming Coefficient Generation

Part 2 için kullanılan yardımcı Python dosyası `generate_hamming_mem.py`'dir. Bu dosya üç temel görev üstlenir: $N = 256$ uzunluğunda Hamming katsayılarını üretmek, katsayıları fixed-point formata çevirmek ve Verilog tarafından okunacak `hamming_q1_15.mem` dosyasını oluşturmak.

Hamming katsayıları floating-point olarak yaklaşık

$$0.08 \leq w_H[n] \leq 1.0 \quad (46)$$

aralığındadır. Donanımda kullanılabilmeleri için bu katsayılar Q1.15 formatına çevrilmiştir. Bu format toplam 16 bitten oluşur; 1 bit işaret/tam kısım, 15 bit ise kesir kısmı için ayrılır. Buna göre 1.0 değeri pratikte yaklaşık 32767 ile temsil edilir. Katsayı dönüşümü

$$w_Q[n] = \text{round}(w_H[n](2^{15} - 1))$$

eşitliği ile yapılır. Window katsayıları pozitif ve 1'den küçük olduğu için Q1.15 formatı bu uygulama için uygundur. Böylece katsayılar 16-bit fixed-point olarak saklanmış ve Verilog tarafında çarpma işlemine hazır hale getirilmiştir.

15 Hardware Architecture Difference from Part 1

Part 1 veri yolu

input samples → AXI packetizer → Xilinx FFT IP → output unpacking → abs_sq →
peak_detector

şeklindeydi. Part 2’de ise veri yolu aşağıdaki hale gelmiştir:

Input samples → Hamming window multiplier → AXI packetizer → Xilinx FFT IP xfft_0
→ output unpacking → abs_sq.v → peak_detector.v → peak_index, peak_value,
peak_valid

Part 2’de FFT IP ve çıkış işleme zinciri değiştirilmemiştir. Değişiklik FFT girişinden önce yapılmıştır. Her örnek, kendi index’ine karşılık gelen Hamming katsayısı ile çarpılmış ve pencere uygulanmış örnek FFT IP’ye gönderilmiştir.

16 Verilog Windowing Implementation

Part 2’de yeni eklenen blok `hamming_window.v`, üst seviye entegrasyon dosyası `fft_windowed_top.v` ve doğrulama ortamı `tb_fft_windowed.sv` etrafında kurulmuştur. `hamming_window.v` bloğunun görevi `hamming_q1_15.mem` dosyasını okumak, frame içindeki sample index’i takip etmek, `in_real` ve `in_imag` değerlerini aynı pencere katsayısı ile çarpmak ve ölçeklenmiş sonucu FFT’ye uygun 12-bit signed biçimde üretmektir.

Her örnek için uygulanan işlem

$$y_{\text{real}}[n] = x_{\text{real}}[n]w_H[n] \quad (47)$$

$$y_{\text{imag}}[n] = x_{\text{imag}}[n]w_H[n] \quad (48)$$

şeklinde. Fixed-point donanım tarafında giriş örneği 12-bit signed, pencere katsayısı ise 16-bit Q1.15 signed olarak ele alınır. Bu nedenle çarpım yaklaşık 28-bit signed bir ara sonuç üretir ve Q1.15 ölçeğini geri almak için 15 bit sağa kaydırma uygulanır:

$$y[n] = \frac{x[n] \cdot w_Q[n]}{2^{15}}$$

Verilog mantığı seviyesinde bu işlem

```

product_real    = in_real * window_coeff;
product_imag    = in_imag * window_coeff;
windowed_real   = product_real >>> 15;
windowed_imag   = product_imag >>> 15;

```

şeklinde gerçekleştirilir. Bu işlem, floating-point dünyadaki $x[n]w_H[n]$ çarpımının fixed-point donanım karşılığıdır. Katsayı Q1.15 formatında olduğu için çarpım sonrası 15 bit sağa kaydırılarak örnek tekrar giriş veri ölçeğine getirilmiştir. Çarpma sonucu bit genişliği artsa da Hamming katsayıları 0 ile 1 arasında olduğu için pencereleme işlemi giriş genişliğini artırmaz; dolayısıyla Part 1'e kıyasla ek bir input overflow riski oluşturmaz.

17 Testbench Differences from Part 1

Part 2 için kullanılan `tb_fft_windowed.sv`, Part 1 testbench'inin pencereleme eklenmiş sürümüdür. Ortak doğrulama omurgası korunmuştur: 200 MHz clock, 256 örneklilik frame'ler, 12.5 MHz ve 10 MHz kompleks girişler, Python `.mem` dosyalarından okunan örnekler ve FFT çıkışından peak/enerji takibi aynıdır. Yeni kontrol noktaları ise pencereleme zincirine odaklanır:

- `hamming_q1_15.mem` dosyasının doğru okunması,
- frame index ile window coefficient index'inin uyumlu ilerlemesi,
- pencere uygulanmış giriş örneklerinin beklenen biçimde küçülmesi,
- 10 MHz için leakage dağılımının Part 1 naive sonuçlarıyla karşılaştırılması,
- peak bin'in yine 13 civarında kalırken komşu bin enerji dağılımının değişmesi.

Hamming window frekans konumunu değiştirmez. Bu nedenle 10 MHz için peak bin'in yine 13 civarında kalması beklenir. Ancak yan lob seviyeleri ve komşu binlere yayılan enerji dağılımı naive FFT'ye göre değişmelidir.

18 Expected Simulation Behavior

Part 2 için başarı kriteri `peak_value`'nin artması değildir. Başarı kriteri leakage'in, özellikle ana lobdan uzak komşu frekans bileşenlerindeki enerji yayılımının, naive FFT'ye göre azalmasıdır. Bu nedenle beklenen davranış aşağıdaki şekilde özetlenebilir:

- 12.5 MHz için peak bin yine 16 civarında kalır.
- 10 MHz için peak bin yine 13 civarında kalır.
- Uzak yan loblarda belirgin azalma görülür.
- Ana lob rectangular window'a göre bir miktar genişler.

- Peak değerleri ve toplam enerji, pencere kazancı nedeniyle Part 1'e göre düşebilir.

Hamming window leakage'i azaltır, ancak bunu ana lob genişliği ve genlik kazancı pahasına yapar. Rectangular window zaman alanında ani başlangıç ve bitişe sahip olduğundan frekans alanında yüksek yan loblar üretir. Hamming window ise frame başı ve sonundaki örnekleri yumuşatarak bu süreksizliği azaltır; sonuç olarak uzak yan loblar bastırılır, fakat ana lob genişliği artar ve frekans çözünürlüğü pratikte bir miktar yumuşar. Bu çerçevede Part 2, Part 1'deki naive FFT altyapısını tekrar anlatmaktan çok, leakage davranışını değiştiren tek yeni elemanın zaman alanı pencereleme olduğunu göstermektedir.

19 Part 2 Simulation Results

Part 2 için alınan `tb_fft_windowed.sv` simülasyon logu aşağıda özetlenmiştir:

```
[158000] === Frame 0: Sending frame: 12.5 MHz windowed ===
[1184000] === Frame 1: Sending frame: 10.0 MHz windowed ===
[2682000] [Frame0 12.5MHz windowed] xk=15 abs_sq=179777
[2686000] [Frame0 12.5MHz windowed] xk=16 abs_sq=988037
[2690000] [Frame0 12.5MHz windowed] xk=17 abs_sq=179777
[3642000] frame 0 total energy = 1347772 (bins=256)
[3646000] peak[0]: index=16 value=988037
[3694000] [Frame1 10.0MHz windowed] xk=12 abs_sq=342325
[3698000] [Frame1 10.0MHz windowed] xk=13 abs_sq=926773
[3702000] [Frame1 10.0MHz windowed] xk=14 abs_sq=76869
[4666000] frame 1 total energy = 1346920 (bins=256)
[4670000] peak[1]: index=13 value=926773
-----
Frame 0 (12.5 MHz windowed): peak bin=16, |X|^2=988037, total=1347772
Frame 1 (10.0 MHz windowed): peak bin=13, |X|^2=926773, total=1346920
Energy ratio (smaller/larger) = 0.999 (tol >= 0.750)
Peak dominance: tone0=187.7x, tone1=176.1x (min=4)
-----
tb_fft_windowed: PASS
```

Bu loglar, Part 2 doğrulamasının beklendiği gibi tamamlandığını göstermektedir. 12.5 MHz windowed frame için `peak_index` = 16, 10 MHz windowed frame için ise `peak_index` = 13 ölçülmüştür. Böylece Hamming window uygulanmasına rağmen tepe frekans bileşeni konumları Part 1 ile aynı kalmıştır. Buna karşılık `peak_value` ve `total_energy` değerleri belirgin biçimde azalmıştır; bu durum Hamming window'un pencere uçlarını bastırmasının doğal sonucudur.

Özellikle 12.5 MHz windowed sonuçta enerji artık tek bir bin üzerinde toplanmamış; `xk_index` = 15, 16 ve 17 çevresine yayılmıştır. Bu davranış, Hamming window'un rectangular window'a göre daha düşük yan loblara fakat daha geniş ana loba sahip olmasıyla uyumludur. Bu nedenle 12.5 MHz tonu hâlâ peak bin olarak 16'da kalırken, enerji 15–17 binleri çevresinde daha geniş bir ana lob oluşturmuştur. 10 MHz windowed sonuçta ise peak yine `xk_index` = 13 üzerinde oluşmuş, fakat

enerji dağılımı yakın komşu binlerde yoğunlaşırken uzak binlerde Part 1’e göre çok daha düşük seviyeye inmiştir. Bu davranış, spectral leakage yan loblarının bastırıldığı şeklinde yorumlanır. Part 2’de başarı kriteri **peak_value**’nın artması değildir. Başarı kriteri leakage’in, özellikle ana lobdan uzak komşu frekans bileşenlerindeki enerji yayılımının, naive FFT’ye göre azalmasıdır.

20 Part 1 and Part 2 Numerical Comparison

Part 2 raporunun Part 1 ile aynı doğrulama kalitesine ulaşabilmesi için peak indeksleri, peak güçleri ve toplam enerji değerleri iki tasarım için yan yana verilmelidir. Hamming window sinyalin frekansını değiştirmedigi için peak bin konumlarının Part 1 ile aynı kalması beklenir. Bu nedenle 12.5 MHz için peak bin yine 16, 10 MHz için peak bin yine 13 civarında oluşmalıdır. Buna karşılık Hamming window sinyalin başını ve sonunu bastırdığı için **peak_value** ve **total_energy** değerleri Part 1’e göre azalabilir.

Sinyal	Tasarım	Teorik bin	Ölçülen peak bin	Peak value	Total energy	Yorum
12.5 MHz	Part 1 Naive FFT	16.0	16	3392965	3392966	Enerji tek bin’de toplandı
12.5 MHz	Part 2 Hamming FFT	16.0	16	988037	1347772	Peak korundu, ana lob 15–17 binlerine yayıldı
10 MHz	Part 1 Naive FFT	12.8	13	2968146	3391331	Leakage gözlemlendi
10 MHz	Part 2 Hamming FFT	12.8	13	926773	1346920	Uzak yan loblar bastırıldı

Tablo 4: Naive FFT ve Hamming windowed FFT sonuçlarının karşılaştırması

Tablodan görüldüğü gibi Hamming window sinyal frekansını değiştirmemiştir; peak bin konumları korunmuştur. 12.5 MHz için peak bin yine 16, 10 MHz için peak bin yine 13 ölçülmüştür. Buna karşılık **peak_value** ve **total_energy** değerleri Part 1’e göre azalmıştır. Bu beklenen bir sonuçtur; çünkü Hamming window zaman domeninde sinyalin başını ve sonunu bastırmaktadır. Dolayısıyla Part 2 başarısı peak değerinin artmasıyla değil, spektral dağılımın özellikle off-bin girişte daha kontrollü hale gelmesiyle değerlendirilmelidir.

21 10 MHz Leakage Bin-by-Bin Comparison

Part 2 raporunun en kritik sayısal doğrulamalarından biri, 10 MHz off-bin tone için komşu frekans bileşenlerindeki güç dağılımının Part 1 ile doğrudan karşılaştırılmasıdır. Aşağıdaki tabloda **bins** 10–18 aralığı için naive FFT ve Hamming windowed FFT sonuçları yan yana verilmiştir.

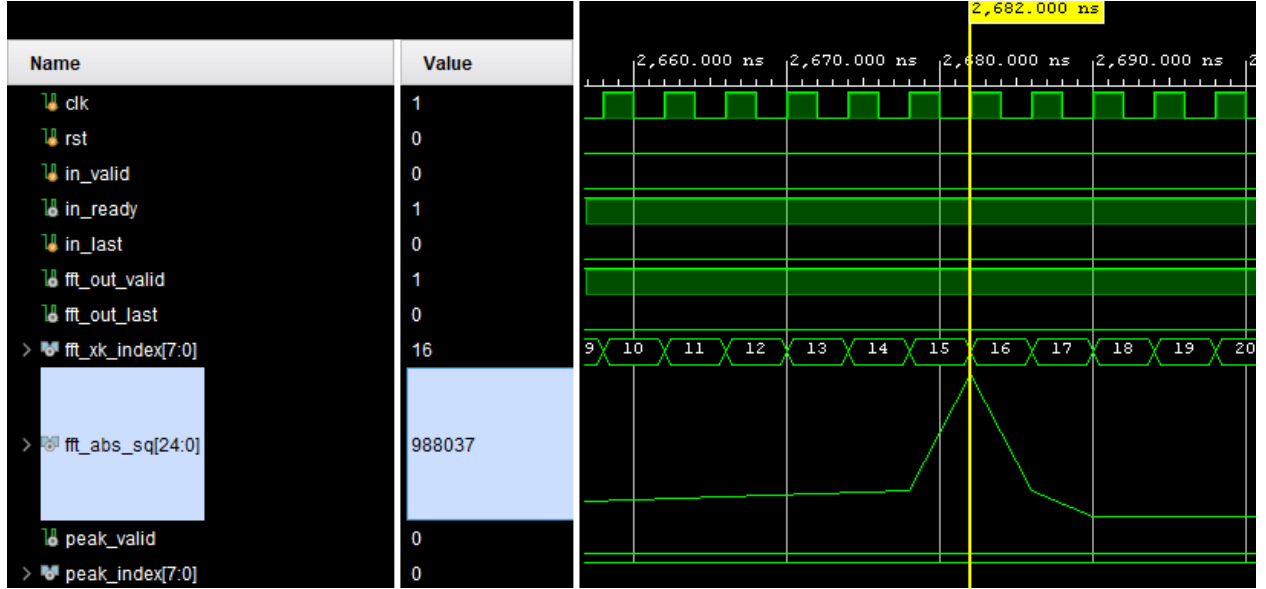
Bin	Part 1 Naive FFT abs_sq	Part 2 Hamming FFT abs_sq	Durum
10	15426	4	Güçlü bastırma
11	36941	549	Bastırıldı
12	185434	342325	Ana lob bölgesi
13	2968146	926773	Peak korundu
14	82645	76869	Ana lob bölgesi
15	24484	45	Bastırıldı
16	11642	8	Bastırıldı
17	6740	18	Bastırıldı
18	4394	18	Bastırıldı

Tablo 5: 10 MHz için naive FFT ve Hamming windowed FFT leakage karşılaştırması

Tablo incelendiğinde Part 1’de 10 MHz sinyalinin enerjisinin yalnızca **xk_index** = 13 üzerinde değil, komşu ve hatta daha uzak frekans bileşenlerine de yayıldığı görülmektedir. Part 2’de Hamming window uygulandıktan sonra peak bin konumu yine 13 üzerinde kalmıştır; ancak özellikle peak’ten uzak bin’lerdeki **abs_sq** değerleri dramatik biçimde azalmıştır. Örneğin **bin** 10 değeri 15426’dan 4’e, **bin** 15 değeri 24484’ten 45’e, **bin** 16 değeri ise 11642’dan 8’e düşmüştür. Buna karşılık **bin** 12 değerinin 185434’ten 342325’e yükselmesi, enerjinin uzak yan loblara yayılmak yerine ana lob çevresinde toplandığını göstermektedir. Bu karşılaştırma, Hamming window’un özellikle peak’ten uzak binlerdeki leakage enerjisini ciddi biçimde düşürdüğünü göstermektedir.

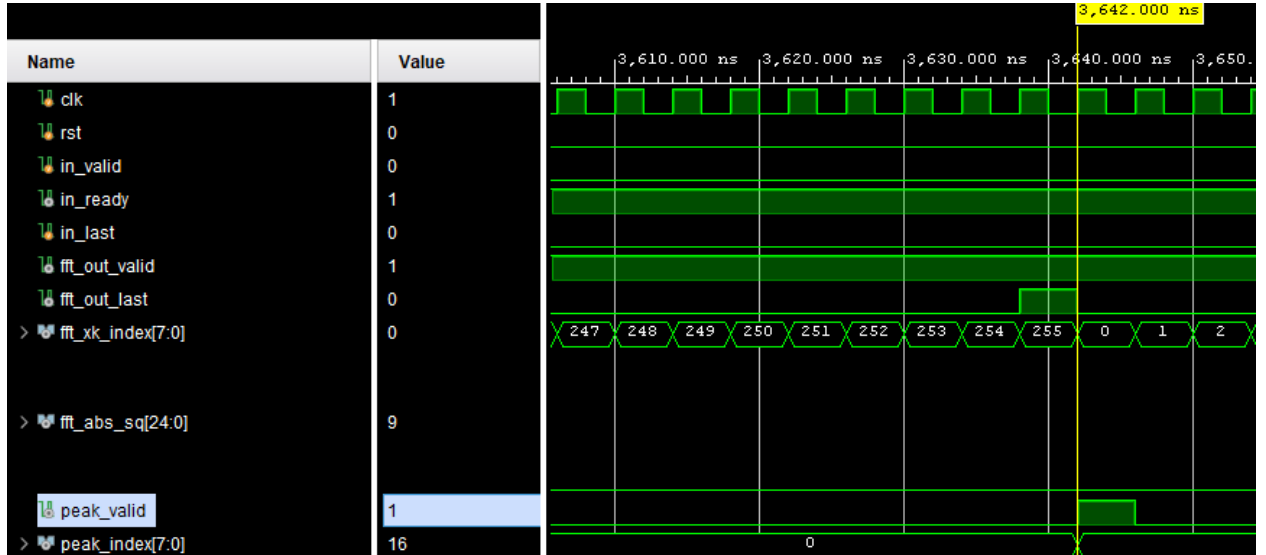
22 Part 2 Waveform Figures

Bu bölümde Part 2 için alınan dört waveform görüntüsü doğrudan rapora eklenmiştir. Part 1’de olduğu gibi bu görseller birlikte incelendiğinde iki kritik sonuç elde edilir: birincisi **fft_abs_sq** yalnızca **fft_out_valid** aktifken anlık **fft_xk_index** ile birlikte anlamlıdır; ikincisi ise **peak_index** ve **peak_value** sinyalleri frame sonunu gösteren olaydan sonra **peak_valid** ile geçerli hale gelmektedir. Ayrıca Part 2’de Part 1’e göre ana lob genişlemesi ve uzak yan lob bastırması waveform üzerinde doğrudan gözlenebilmektedir.



Şekil 6: 12.5 MHz windowed frame için anlık FFT çıkışı ve `fft_abs_sq` güncellemesi

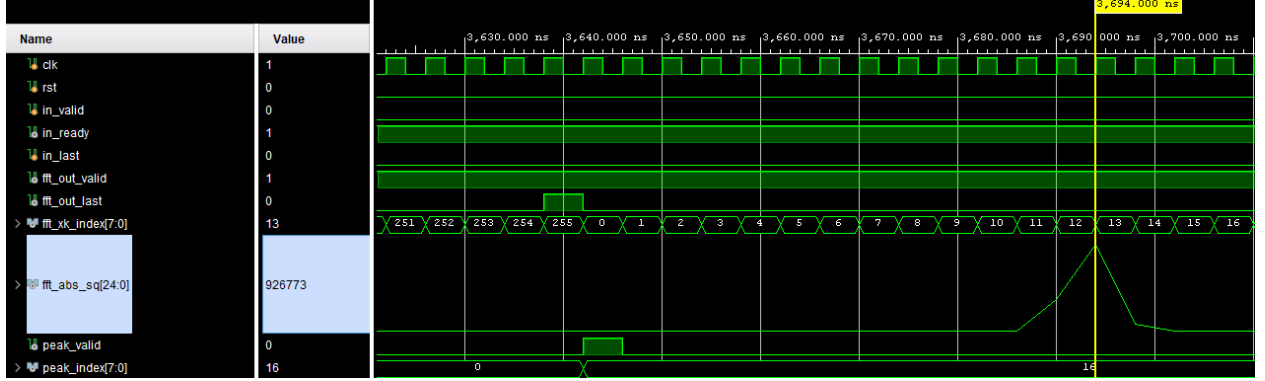
Şekil 6’de cursor `fft_xk_index = 16` anına yerleştirilmiştir ve bu anda `fft_abs_sq = 988037` ölçülmektedir. Bu değer, simülasyon logundaki `peak[0]: index=16 value=988037` sonucu ile tam uyumludur. Bununla birlikte Part 1’deki naive FFT sonucundan farklı olarak enerji yalnızca tek bir bin üzerinde toplanmamıştır; aynı frame içinde `xk_index = 15` ve `17` için de `abs_sq = 179777` görülmektedir. Teorik olarak 12.5 MHz tonu hâlâ tam olarak 16. bin’e karşılık gelmektedir; ancak Hamming window ana lobu genişlettiği için ideal rectangular-window sonucundaki tek-bin yoğunlaşması yerine 15–17 binleri çevresinde daha geniş bir spektral tepe oluşmuştur.



Şekil 7: 12.5 MHz windowed frame sonunda `peak_valid` ve `peak_index` üretimi

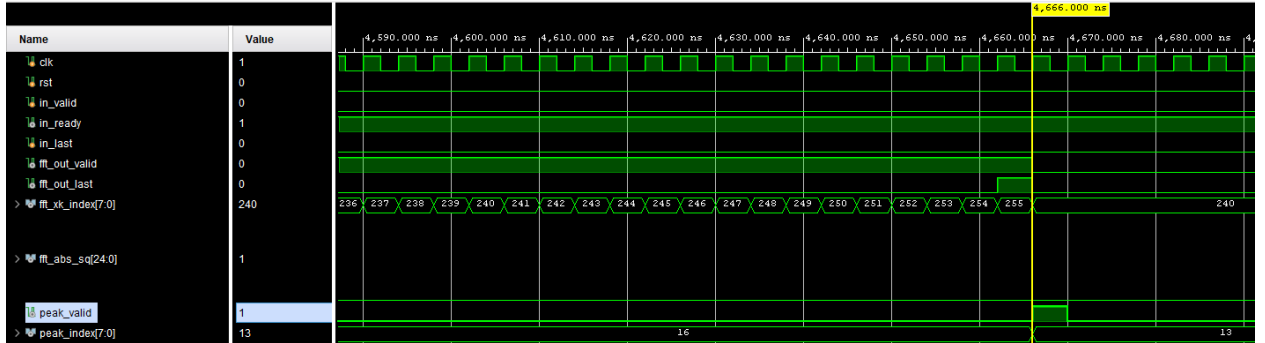
Şekil 7’de frame sonuna yakın bölgede `fft_out_last` olayını takiben `peak_valid` darbesi oluşmaktadır. Bu anda `peak_index = 16` raporlanmakta ve logda bildirilen `peak_value = 988037` değeri

bu peak olayına karşılık gelmektedir. Görsel üzerinde `fft_xk_index` hattının artık 255'ten sonra sıfıra dönen akış bilgisini taşıdığı görülmektedir; dolayısıyla frame özeti olan `peak_index` ile aynı anda aynı değeri göstermesi gerekmez. Bu da Part 1'de vurgulanan valid/zamanlama yorumunun Part 2 için de aynı şekilde geçerli olduğunu gösterir.



Şekil 8: 10 MHz windowed frame için anlık FFT çıkışı ve leakage davranışı

Şekil 8'de cursor `fft_xk_index` = 13 anına yerleştirilmiştir ve bu anda `fft_abs_sq` = 926773 ölçülmektedir. Bu değer, 10 MHz windowed frame için logda verilen tepe güç sonucu ile birebir örtüşmektedir. Aynı görselde `peak_index` hattının hâlâ bir önceki frame'den kalan 16 değerini tuttuğu görülmektedir; çünkü `peak_valid` = 0 iken bu sinyal yeni frame için henüz anlamlı değildir. Bu durum hata değildir; anlık FFT akışı ile frame sonu özeti farklı zamanlarda geçerli hale gelmektedir. Teorik olarak 10 MHz sinyalinin bin karşılığı 12.8 olduğundan peak'in 13 civarında oluşması beklenmektedir ve waveform bu beklentiyi doğrulamaktadır. Part 1'e göre önemli fark, uzak yan lob enerjilerinin çok daha düşük olmasıdır; bu durum Tablo 5'de sayısal olarak da görülmektedir.



Şekil 9: 10 MHz windowed frame sonunda `peak_valid` ve `peak_index` üretimi

Şekil 9'de ikinci frame tamamlandıktan sonra `peak_valid` darbesi oluşmakta ve `peak_index` = 13 değeri raporlanmaktadır. Log çıktısına göre bu olaya karşılık gelen tepe güç değeri `peak_value` = 926773'tür. Aynı anda `fft_xk_index` hattı artık geçerli FFT akış bilgisini taşımadığından 240 gibi stale bir değerde kalabilmektedir. Bu davranış Part 1'de tartışılan AXI-Stream yorum kuralı ile birebir aynıdır: `fft_xk_index` ve `fft_abs_sq` yalnızca `fft_out_valid` = 1 iken anlamlıdır; `peak_index` ve `peak_value` ise yalnızca `peak_valid` = 1 iken frame özeti olarak anlam kazanır.

Bu dört waveform birlikte değerlendirildiğinde Part 2 için genel tablo netleşmektedir: Hamming window sinyalin frekansını değiştirmedığı için peak bin konumları korunmuştur; 12.5 MHz için peak yine 16, 10 MHz için ise yine 13 civarında oluşmaktadır. Buna karşılık spektrumun şekli Part 1’e göre belirgin biçimde değişmiştir. 12.5 MHz tone için ana lob genişlemiş, 10 MHz tone için ise peak çevresindeki yapı korunurken uzak yan lob seviyeleri ciddi biçimde bastırılmıştır. Kısacası waveform görselleri, log çıktıları ve teorik beklentiler birlikte değerlendirildiğinde Part 2 tasarımının hem beklenen peak konumlarını koruduğu hem de leakage yan loblarını azalttığı açıkça doğrulanmaktadır.

23 Part 2 Conclusion

Part 2 sonucunda Hamming window entegrasyonu başarılı şekilde doğrulanmıştır. Part 1’de kullanılan naive FFT veri yolu korunmuş, yalnızca FFT IP girişine `hamming_window.v` bloğu eklenmiştir. Bu blok Python tarafından üretilen Q1.15 formatındaki Hamming katsayılarını `$readmemh` ile okuyarak her giriş örneğini ilgili pencere katsayısı ile çarpmaktadır.

Simülasyon doğrulaması `tb_fft_windowed.sv` testbench’i ile yapılmıştır. Testbench 200 MHz clock altında iki ardışık frame göndermiştir: 12.5 MHz windowed complex tone ve 10 MHz windowed complex tone. Her iki frame için `peak_index`, `peak_value` ve `total_energy` değerleri kaydedilmiş; ayrıca 10 MHz için `bins 10-18` dağılımı Part 1 ile yan yana karşılaştırılmıştır.

Part 1 ile karşılaştırıldığında peak bin konumları değişmemiştir; çünkü Hamming window giriş sinyalinin frekansını değiştirmemektedir. 12.5 MHz için peak bin yine 16, 10 MHz için peak bin yine 13 olarak ölçülmüştür. Ancak Part 2’de spektral dağılım belirgin biçimde değişmiştir. 12.5 MHz tone için enerji 15–17 binleri çevresinde daha geniş bir ana loba yayılmıştır. 10 MHz tone için ise peak yine 13. bin üzerinde kalmış, fakat uzak yan lob enerjileri keskin biçimde bastırılmıştır. Part 2’de başarı kriteri `peak_value`’nın artması değildir. Başarı kriteri leakage’in, özellikle ana lobdan uzak komşu frekans bileşenlerindeki enerji yayılımının, naive FFT’ye göre azalmasıdır.

Sayısal olarak 12.5 MHz windowed frame için `peak_value = 988037` ve `total_energy = 1347772`, 10 MHz windowed frame için ise `peak_value = 926773` ve `total_energy = 1346920` ölçülmüştür. Enerji oranının 0.999 olması, iki tone’un pencereleme sonrasında da tutarlı genlik yapısına sahip olduğunu göstermektedir. Peak dominance değerleri 187.7x ve 176.1x olarak ölçülmüş, testbench PASS sonucu vermiştir. Toplanan sayısal sonuçlar, zaman-domain windowing işleminin FPGA üzerinde fixed-point katsayılar ve Verilog veri yolu ile tutarlı biçimde uygulanabildiğini ortaya koymaktadır. Bu nedenle Part 2’de elde edilen windowed FFT çıktıları, Part 3’te kurulacak frequency-domain three-term windowing bloğunun doğrulanmasında referans olarak kullanılacaktır.

24 Part 3: Frequency-Domain Windowing'in Amacı

Part 3'ün amacı, Part 2'de zaman domeninde uygulanan Hamming window işleminin frekans domenindeki eşdeğerini donanım üzerinde gerçekleştirmektir. Part 2'de giriş dizisi önce pencere ile çarpılmış, ardından FFT alınmıştır:

$$x_w[n] = x[n] \cdot w[n]$$

$$X_w[m] = \mathcal{F}\{x_w[n]\}$$

Part 3'te ise yaklaşım ters sıradadır. Önce window uygulanmamış sinyalin FFT'si alınır:

$$X[m] = \mathcal{F}\{x[n]\}$$

Ardından Hamming window etkisi FFT çıkışı üzerinde üç terimli bir frekans domeni işlemi ile oluşturulur. Böylece Part 3, Part 1'deki window'suz FFT akışını korurken, Part 2'deki leakage azaltıcı pencere etkisini FFT sonrasında üretmeyi hedefler. Bu yönüyle bölüm, Part 1 ve Part 2 ile doğrudan ilişkili olmakla birlikte, frekans domeninde windowing gerçekleştiren bağımsız bir donanım çözümü ortaya koymaktadır.

24.1 Teorik Altyapı

Zaman domeninde çarpma işlemi frekans domeninde convolution'a karşılık gelir. Bu nedenle bir pencere fonksiyonunun zaman domeninde uygulanması, FFT çıktı binlerinin frekans domeninde komşu terimlerle yeniden ağırlıklandırılması olarak yorumlanabilir. Part 2'de uygulanan Hamming pencereleme için temel ilişki şu şekildedir:

$$x_w[n] = x[n] \cdot w[n]$$

$$X_w[m] = \mathcal{F}\{x_w[n]\}$$

Part 3'te hedef, doğrudan $X_w[m]$ 'yi zaman domeninde değil, FFT çıkışındaki $X[m]$ binlerinden üretmektir. Bu eşdeğerlik, cosine-window ailesinin frekans domeninde az sayıda komşu bin ile temsil edilebilmesi sayesinde mümkündür. Böylece Hamming window etkisi, her FFT çıkış bininin yalnızca kendisi ve iki komşusu kullanılarak yeniden hesaplanmasıyla elde edilir.

24.2 Hamming Window'un Frekans Domenindeki Üç Terimli Karşılığı

Hamming window genel cosine window ailesinin bir üyesidir ve

$$w[n] = \alpha - \beta \cos\left(\frac{2\pi n}{N}\right)$$

şeklinde ifade edilir. Hamming window için katsayılar

$$\alpha = 0.54$$

$$\beta = 0.46$$

$$\frac{\beta}{2} = 0.23$$

olarak alınır. Euler bağıntısı kullanıldığında

$$\cos\left(\frac{2\pi n}{N}\right) = \frac{1}{2} \left(e^{j2\pi n/N} + e^{-j2\pi n/N} \right)$$

elde edilir. Bu ifade Hamming window denkleminde yerine yazıldığında

$$w[n] = \alpha - \frac{\beta}{2} e^{j2\pi n/N} - \frac{\beta}{2} e^{-j2\pi n/N}$$

olur. Dolayısıyla zaman domeninde pencere uygulanmış sinyal

$$x_w[n] = \alpha x[n] - \frac{\beta}{2} x[n] e^{j2\pi n/N} - \frac{\beta}{2} x[n] e^{-j2\pi n/N}$$

şeklinde üç terime ayrılır. DFT frekans kaydırma özelliği kullanılarak

$$x[n] e^{j2\pi n/N} \iff X[m-1]$$

$$x[n] e^{-j2\pi n/N} \iff X[m+1]$$

eşdeğerlikleri elde edilir. Böylece frekans domenindeki üç terimli ifade

$$Y[m] = \alpha X[m] - \frac{\beta}{2} X[m-1] - \frac{\beta}{2} X[m+1]$$

şeklinde dir. Hamming için özel form doğrudan

$$Y[m] = 0.54X[m] - 0.23X[m-1] - 0.23X[m+1]$$

olarak yazılır. Bu sonuç, Hamming window'un frekans domeninde her bin için yalnızca üç FFT çıkışına ihtiyaç duyduğunu gösterir:

$$X[m-1], \quad X[m], \quad X[m+1]$$

Dolayısıyla Part 3'te gerçekleştirilen işlem, genel bir convolution yerine, Hamming pencerenin özel yapısı sayesinde üç komşulu yerel bir yeniden ağırlıklandırma'dır. Bu denklem, Part 2'de zaman domeninde Hamming window uygulanıp sonra FFT alınması ile Part 3'te önce window'suz FFT alınıp sonra frekans domeninde üç terimli işlem uygulanmasının teorik olarak aynı sonucu verdiğini gösterir. Bu yüzden Part 3'ün amacı yeni bir window türü üretmek değil, Part 2'deki Hamming window etkisini FFT çıkışında yeniden oluşturmaktır.

24.3 Real ve Imaginary Bileşenlere Uygulama

FFT çıkışı kompleks olduğundan her bin

$$X[m] = a_m + jb_m$$

$$X[m-1] = a_{m-1} + jb_{m-1}$$

$$X[m+1] = a_{m+1} + jb_{m+1}$$

şeklinde yazılır. Burada a_m FFT çıkışının real kısmını, b_m ise imaginary kısmını temsil eder. Üç terimli frekans domeni işlemi bu nedenle real ve imaginary bileşenlere ayrı ayrı uygulanmalıdır:

$$Y_{\text{real}}[m] = 0.54a_m - 0.23a_{m-1} - 0.23a_{m+1}$$

$$Y_{\text{imag}}[m] = 0.54b_m - 0.23b_{m-1} - 0.23b_{m+1}$$

Bu ayrım, `three_term.v` içinde complex FFT çıkışının iki bağımsız signed veri yolu olarak işlenmesini sağlar. `three_term.v` modülü FFT IP'den gelen real ve imaginary bileşenleri ayrı ayrı işler. Real kısım için aynı üç terimli formül, imaginary kısım için de aynı üç terimli formül uygulanır. Sonrasında yeniden üretilen $Y_{\text{real}}[m]$ ve $Y_{\text{imag}}[m]$ değerleri birlikte pencere uygulanmış kompleks FFT sonucunu temsil eder ve `y_re_out` ile `y_im_out` sinyalleri üzerinde gözlenir.

24.4 Circular Boundary Koşulları

FFT frekans binleri periyodik ve circular kabul edildiği için komşuluk ilişkisi uçlarda devirli olarak uygulanmalıdır. Bu nedenle sınır koşulları aşağıdaki gibidir:

$$X[m-1] = X[(m-1) \bmod N]$$

$$X[m+1] = X[(m+1) \bmod N]$$

$$m = 0 \Rightarrow X[m-1] = X[N-1]$$

$$m = N-1 \Rightarrow X[m+1] = X[0]$$

Bu circular komşuluk uygulanmadığında, Part 2’de zaman domeninde Hamming window uygulanarak elde edilen sonuç ile Part 3’te frekans domeninde elde edilen sonuç birebir uyuşmaz. Dolayısıyla Part 3’te teorik eşdeğerliğin sağlanması için boundary işlemi opsiyonel değil, zorunludur.

24.5 Verilog Tasarım Mantığı

Part 3 veri akışında önce Xilinx FFT IP, window uygulanmamış giriş sinyalinin FFT’sini üretir. FFT çıkışındaki her bin için real, imaginary ve `xk_index` bilgisi `three_term.v` modülüne gönderilir. Ancak modül, her m bin’i için $X[m-1]$, $X[m]$ ve $X[m+1]$ değerlerine ihtiyaç duyduğundan FFT çıkışını doğrudan anlık olarak işleyemez. Bunun yerine önce tüm 256 binlik frame dahili belleğe kaydedilir.

Bu amaçla modül içinde iki ayrı bellek yapısı kullanılmaktadır:

- `xre_mem`: FFT real bileşenlerini saklar.
- `xim_mem`: FFT imaginary bileşenlerini saklar.

Bu mimari, Part 1 ve Part 2’deki akışa göre farklıdır. Part 1 ve Part 2’de FFT çıkışı doğrudan `abs_sq.v` modülüne gönderilebilirken, Part 3’te pencere uygulanmış frekans bininin üretilebilmesi için önce tüm frame’in toplanması gerekir. Dolayısıyla Part 3 doğası gereği frame tabanlı çalışan ek bir ara işleme katmanı içerir.

Aşağıdaki kısa blok akış Part 3 entegrasyonunu özetlemektedir:

```

FFT IP output
  ↓
three_term.v
  ↓
abs_sq.v
  ↓
peak_detector.v
  ↓
peak_index, peak_value, peak_valid

```

24.6 three_term.v Modülünün Çalışma Akışı

three_term.v modülü iki fazlı bir FSM mantığı ile çalışır:

1. S_CAPTURE fazı,
2. S_SWEEP fazı.

S_CAPTURE fazında FFT IP'den gelen $X[0]$ ile $X[255]$ arasındaki tüm binler RAM'e yazılır. Bu sırada `xk_index_in` adres olarak kullanılır. `xk_valid_in` aktif olduğunda `xk_re_in` ve `xk_im_in` ilgili adrese kaydedilir. Frame'in son bin'i ise `xk_last_in` sinyali ile anlaşılır.

S_SWEEP fazında modül $m = 0$ ile $m = N - 1$ arasında dolaşır. Her m için RAM'den üç değer okunur:

$$X[m - 1], \quad X[m], \quad X[m + 1]$$

Ardından Hamming'in üç terimli frekans domeni formülü real ve imaginary taraf için ayrı ayrı uygulanır:

$$Y[m] = 0.54X[m] - 0.23X[m - 1] - 0.23X[m + 1]$$

Böylece modül her sweep çevriminde yeni bir pencere uygulanmış frekans bin'i üretir. Burada katsayılar floating-point olarak kullanılmaz; fixed-point formatta temsil edilir. Hanning window'da 0.5 ve 0.25 gibi katsayılar bit shift ile uygulanabilirken, Hamming window'daki 0.54 ve 0.23 katsayıları iki'nin kuvveti olmadığından genel fixed-point çarpma gerektirir. Bu nedenle Part 3'te yalnızca kaydırma tabanlı bir komşu bin toplama mantığı yeterli değildir; gerçek multiplier tabanlı bir ağırlıklandırma gerekir.

Part 3'ün Part 1 ve Part 2'ye göre neden ek gecikme oluşturduğu da bu noktada ortaya çıkar. three_term.v her output bin için komşu binlere ihtiyaç duyduğu için önce tüm 256 binlik frame'i yakalamak, ardından ikinci bir sweep ile 256 adet yeni frekans domeni sonucu üretmek zorundadır. 200 MHz clock altında 256 cycle yaklaşık olarak

$$256 \times 5 \text{ ns} = 1280 \text{ ns} = 1.28 \mu s$$

ek sweep süresi oluşturur. Buna birkaç pipeline veya register cycle daha eklenebilir. Dolayısıyla Part 3'te gözlenen ek latency milisaniye değil, mikro saniye mertebesinde.

24.7 abs_sq ve peak_detector ile Entegrasyon

three_term.v tarafından üretilen pencere uygulanmış kompleks frekans binleri, Part 1 ve Part 2'de olduğu gibi yine abs_sq.v modülüne aktarılır. Bu aşamada her bin için büyüklük karesi hesaplanır:

$$|Y[m]|^2 = Y_{\text{real}}^2[m] + Y_{\text{imag}}^2[m]$$

Bundan sonra `peak_detector.v` modülü bir frame boyunca maksimum güç değerini izleyerek `peak_index`, `peak_value` ve `peak_valid` sinyallerini üretir. Böylece Part 3'te pencereleme katmanı FFT sonrasına taşınmış olsa da, güç hesabı ve peak tespiti zinciri Part 1 ve Part 2 ile uyumlu kalır. Bu seçim, üç bölüm arasında karşılaştırmalı doğrulama yapılmasını kolaylaştırır.

Beklenen peak doğrulaması teorik olarak Part 1 ve Part 2 ile aynıdır. 12.5 MHz için

$$k = \frac{12.5 \text{ MHz}}{200 \text{ MHz}/256} = 16$$

olduğundan beklenen peak bin

$$k = 16$$

şeklindedir. 10 MHz için ise

$$k = \frac{10 \text{ MHz}}{200 \text{ MHz}/256} = 12.8$$

olduğundan beklenen peak yaklaşık olarak

$$k \approx 13$$

olmalıdır.

Giriş Sinyali	Teorik Bin	Beklenen Peak
12.5 MHz	16.0	16
10.0 MHz	12.8	13

Tablo 6: Part 3 için teorik bin ve beklenen peak karşılaştırması

24.8 Testbench Düzeltmesi ve Frame Zamanlaması

Part 3 testbench'inde başlangıçta iki giriş frame'i arka arkaya gönderildiğinde ikinci frame için peak sonucu oluşmamıştır. Bunun sebebi FFT IP'nin ikinci frame'i işlememesi değildir. Xilinx FFT IP `Pipelined Streaming I/O` modunda çalışmaktadır ve teorik olarak ardışık frame kabul edebilir. Sorun, `three_term.v` modülünün tek-buffer ve iki fazlı capture/sweep FSM yapısından kaynaklanmaktadır.

Eğer Frame 1, Frame 0 için `three_term.v` modülü hâlâ `S_SWEEP` durumundayken FFT çıkışına ulaşırsa, Frame 1'in `frame_end` bilgisi FSM tarafından kaçırılabilir. Bu durumda Frame 1 RAM'e

yazılmış olsa bile yeni sweep başlatılamaz. Sonuç olarak ikinci `peak_valid` oluşmaz ve `peak_index` Frame 0 değerinde kalır.

Bu nedenle testbench'te iki frame arka arkaya gönderilmemiştir. Önce Frame 0 gönderilmiş, ardından Frame 0'a ait peak sonucu ve enerji hesabı beklenmiş, daha sonra Frame 1 uygulanmıştır. Düzeltilmiş stimulus mantığı aşağıdaki gibidir:

```
send_frame("12.5 MHz freqwin", 0, tone12_re, tone12_im);

wait (peak_count >= 1 && frame_energy_idx >= 1);
repeat (20) @(posedge clk);

send_frame("10.0 MHz freqwin", 1, tone10_re, tone10_im);

wait (peak_count >= 2 && frame_energy_idx >= 2);
```

Bu değişiklik FFT IP'nin Pipelined Streaming I/O ayarını değiştirmez. Sadece Part 3'te kullanılan tek-buffer `three_term` yapısına uygun testbench stimulus zamanlaması sağlar. Daha profesyonel bir gerçek zamanlı streaming çözümde `three_term.v` modülüne input ready/backpressure eklenebilir veya ping-pong buffer kullanılabilir. Ping-pong buffer mimarisinde bir buffer Frame 0 için sweep yaparken diğer buffer Frame 1'i yakalayabilir. Ancak bu çalışma kapsamında frame'leri testbench düzeyinde ayırmak yeterli ve anlaşılır bir çözümdür.

24.9 Part 3 Simulation Results

Part 3 için alınan `tb_fft_freqwin` simülasyon logu aşağıda özetlenmiştir:

```
[158000] === Frame 0: Sending frame: 12.5 MHz freqwin ===
[3698000] [Frame0 12.5MHz freqwin] xk=10 abs_sq=0
[3702000] [Frame0 12.5MHz freqwin] xk=11 abs_sq=0
[3706000] [Frame0 12.5MHz freqwin] xk=12 abs_sq=0
[3710000] [Frame0 12.5MHz freqwin] xk=13 abs_sq=0
[3714000] [Frame0 12.5MHz freqwin] xk=14 abs_sq=0
[3718000] [Frame0 12.5MHz freqwin] xk=15 abs_sq=179776
[3722000] [Frame0 12.5MHz freqwin] xk=16 abs_sq=990026
[3726000] [Frame0 12.5MHz freqwin] xk=17 abs_sq=179776
[3730000] [Frame0 12.5MHz freqwin] xk=18 abs_sq=0
[4678000] frame 0 total energy = 1349579 (bins=256)
[4682000] peak[0]: index=16 value=990026
[4762000] === Frame 1: Sending frame: 10.0 MHz freqwin ===
[8302000] [Frame1 10.0MHz freqwin] xk=10 abs_sq=2
[8306000] [Frame1 10.0MHz freqwin] xk=11 abs_sq=557
[8310000] [Frame1 10.0MHz freqwin] xk=12 abs_sq=342065
[8314000] [Frame1 10.0MHz freqwin] xk=13 abs_sq=927197
[8318000] [Frame1 10.0MHz freqwin] xk=14 abs_sq=76745
[8322000] [Frame1 10.0MHz freqwin] xk=15 abs_sq=41
```

```

[8326000] [Frame1 10.0MHz freqwin] xk=16 abs_sq=13
[8330000] [Frame1 10.0MHz freqwin] xk=17 abs_sq=20
[8334000] [Frame1 10.0MHz freqwin] xk=18 abs_sq=13
[9282000] frame 1 total energy = 1346953 (bins=256)
[9286000] peak[1]: index=13 value=927197
-----
Frame 0 (12.5 MHz freqwin): peak bin=16, |X|^2=990026, total=1349579
Frame 1 (10.0 MHz freqwin): peak bin=13, |X|^2=927197, total=1346953
Energy ratio (smaller/larger) = 0.998 (tol >= 0.750)
Peak dominance: tone0=187.8x, tone1=176.2x (min=4)
NOTE: compare freqwin dom1 (176.2x) vs naive dom1 to confirm three-term windowing benefit.
-----
tb_fft_freqwin: PASS

```

Bu sonuçlar Part 3 frequency-domain windowing tasarımının beklediği gibi çalıştığını göstermektedir. 12.5 MHz giriş için teorik bin değeri 16’dır ve simülasyonda **peak_index = 16**, **peak_value = 990026** ölçülmüştür. 10 MHz giriş için teorik bin değeri 12.8 olduğundan peak’ın 13 civarında oluşması beklenir; simülasyonda **peak_index = 13**, **peak_value = 927197** olarak bulunmuştur. Böylece Part 3, Part 2’deki Hamming windowed FFT ile aynı peak bin davranışını üretmiştir.

Frame 0 toplam enerji

$$E_0 = 1349579$$

ve Frame 1 toplam enerji

$$E_1 = 1346953$$

olarak ölçülmüştür. Enerji oranı

$$\frac{\min(E_0, E_1)}{\max(E_0, E_1)} = 0.998$$

şeklindedir. Bu oran iki giriş frame’inin tutarlı genlik ve enerji seviyelerinde işlendiğini gösterir. Peak dominance tanımı da Part 1 ve Part 2 ile aynı şekilde korunmuştur:

$$D = \frac{\max_m |Y[m]|^2}{E_{\text{total}}/N}$$

Part 3 için dominance değerleri

$$D_{12.5\text{MHz}} = 187.8$$

$$D_{10\text{MHz}} = 176.2$$

olarak ölçülmüştür. Bu değerler testbench'te belirlenen minimum eşik olan 4'ün çok üzerindedir. Bu nedenle her iki frame'de de belirgin bir spektral peak bulunduğu doğrulanmıştır.

Frame	Giriş Sinyali	Teorik Bin	Ölçülen Peak Bin	Peak Value	Total Energy	Peak Dominance	Yorum
0	12.5 MHz freqwin	16.0	16	990026	1349579	187.8x	Peak 16'da, Hamming ana lobu 15–17 çevresinde
1	10.0 MHz freqwin	12.8	13	927197	1346953	176.2x	Peak 13'te, uzak yan loblar bastırılmış

Tablo 7: Part 3 frequency-domain windowing simülasyon sonuçlarının özeti

24.10 Part 3 Numerical Comparison with Part 2

Part 2 time-domain Hamming windowing sonuçları ile Part 3 frequency-domain three-term windowing sonuçları doğrudan karşılaştırıldığında iki yaklaşımın donanım üzerinde birbirine çok yakın değerler ürettiği görülmektedir. Part 2'de 12.5 MHz için `peak_bin` = 16, `peak_value` = 988037, `total_energy` = 1347772; 10 MHz için ise `peak_bin` = 13, `peak_value` = 926773, `total_energy` = 1346920 ölçülmüştü. Part 3'te karşılık gelen sonuçlar 12.5 MHz için `peak_bin` = 16, `peak_value` = 990026, `total_energy` = 1349579; 10 MHz için ise `peak_bin` = 13, `peak_value` = 927197, `total_energy` = 1346953 olarak bulunmuştur.

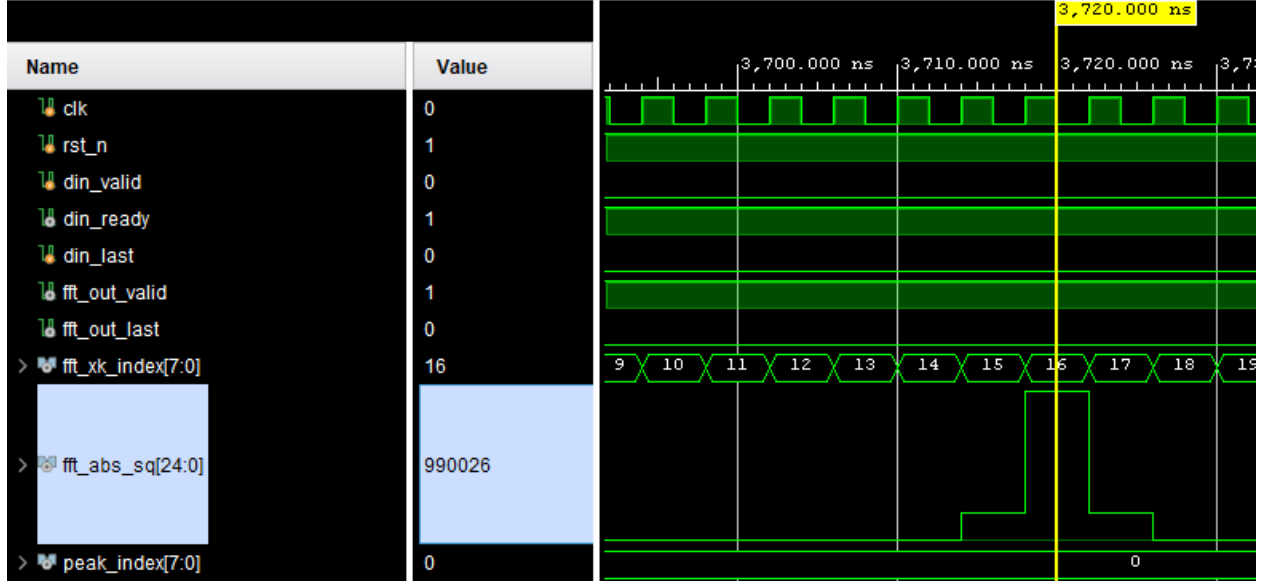
Giriş Sinyali	Part 2 Peak	Part 3 Peak	Part 2 Peak Value	Part 3 Peak Value	Part 2 Total Energy	Part 3 Total Energy	Yorum
12.5 MHz	16	16	988037	990026	1347772	1349579	Sonuçlar çok yakın, peak konumu aynı
10.0 MHz	13	13	926773	927197	1346920	1346953	Sonuçlar çok yakın, three-term eşdeğerlik doğrulandı

Tablo 8: Part 2 time-domain Hamming ile Part 3 frequency-domain windowing sonuçlarının karşılaştırması

Part 2 ve Part 3 sonuçları hem peak bin hem de güç değerleri açısından oldukça yakındır. Küçük farklar fixed-point katsayı yuvarlama, FFT IP scaling, `three_term.v` modülündeki rounding/saturation ve donanımsal bit genişliği tercihleri nedeniyle oluşabilir. Önemli olan peak bin konumlarının aynı kalması ve toplam enerji ile peak güç değerlerinin birbirine yakın olmasıdır. Bu sonuçlar, frequency-domain three-term windowing yönteminin time-domain Hamming windowing ile donanımsal olarak tutarlı olduğunu göstermektedir.

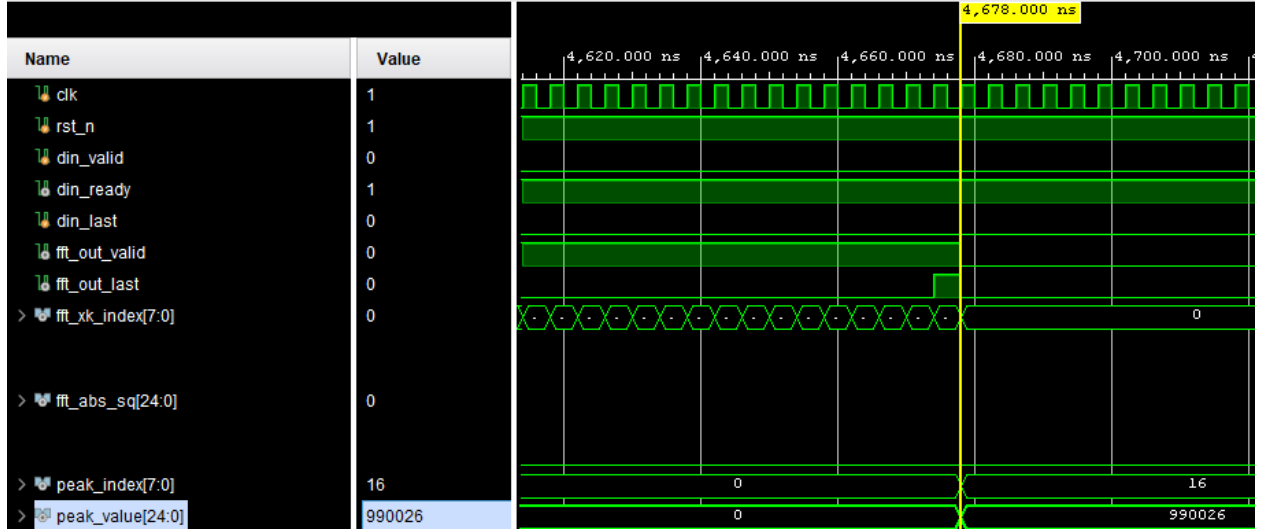
24.11 Part 3 Waveform Analysis

Part 3 için alınan dört waveform görseli, hem three-term frekans domeni işleminin anlık davranışını hem de frame sonunda `peak_detector.v` tarafından üretilen özet sonuçları göstermektedir.



Şekil 10: Part 3 — 12.5 MHz freqwin frame için anlık FFT çıkışı ve three-term sonucu

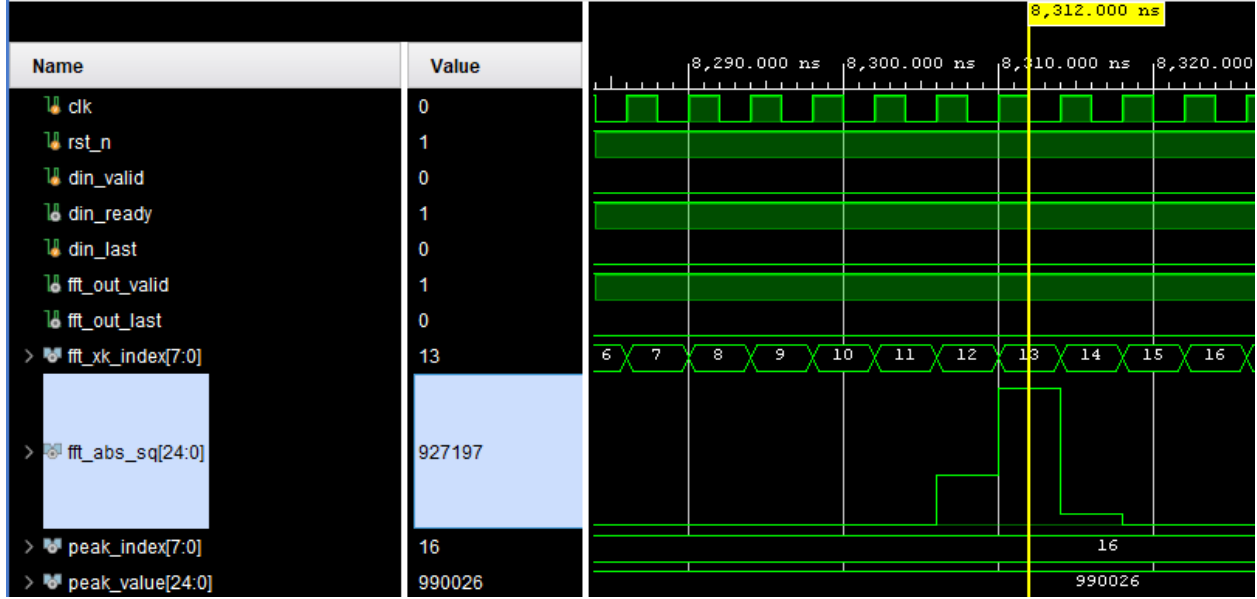
Şekil 10’de cursor `fft_xk_index` = 16 anına yerleştirilmiştir. `fft_out_valid` = 1 iken `fft_abs_sq` = 990026 ölçülmektedir. Bu değer simülasyon logundaki [Frame0 12.5MHz freqwin] `xk=16 abs_sq=990026` satırı ile uyumludur. 12.5 MHz sinyali teorik olarak 16. bin’e karşılık geldiği için maksimum güç bu bin üzerinde oluşmuştur. Bununla birlikte Hamming eşdeğer üç terimli işlem nedeniyle enerji yalnızca tek bin’de kalmamış, 15 ve 17 komşu binlerinde de `abs_sq` = 179776 görülmüştür. Bu durum Hamming window’un ana lobu genişletmesiyle uyumludur.



Şekil 11: Part 3 — 12.5 MHz freqwin frame sonunda peak üretimi

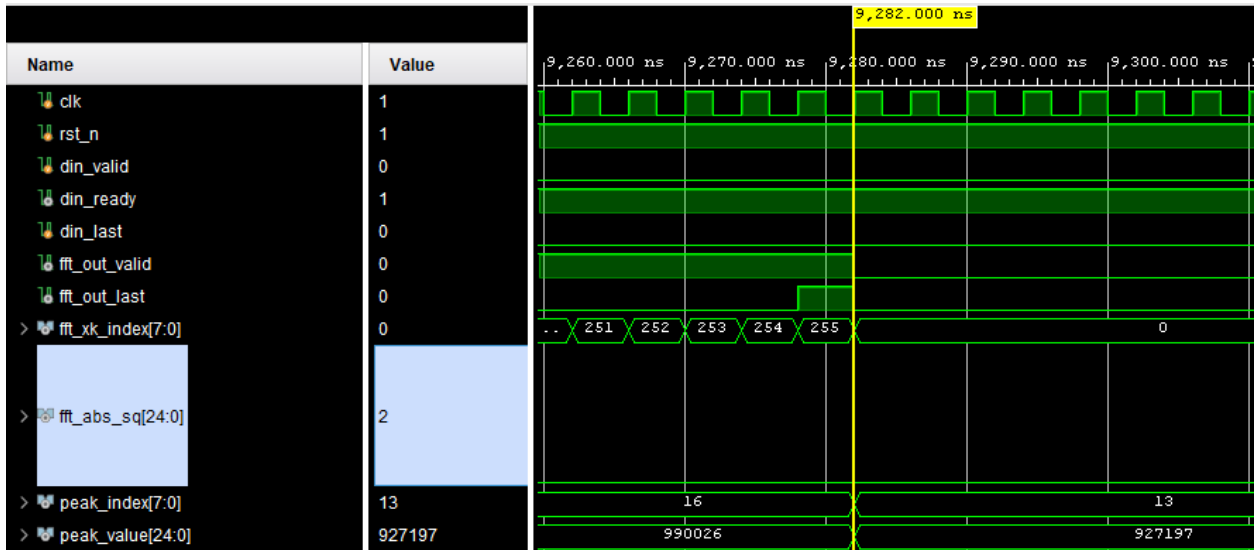
Şekil 11’de Frame 0 tamamlandıktan sonra `peak_valid` darbesi oluşmakta ve `peak_index` = 16, `peak_value` = 990026 değerleri register’a alınmaktadır. Bu sonuç simülasyon logundaki `peak[0]: index=16 value=990026` satırıyla birebir uyumludur. Görsel, anlık FFT akış bilgisi olan `fft_xk_index`

ile frame sonunda üretilen özet sonuç olan **peak_index** sinyallerinin farklı zamanlarda anlamlı olduğunu göstermektedir.



Şekil 12: Part 3 — 10 MHz freqwin frame için anlık FFT çıkışı ve leakage davranışı

Şekil 12’de cursor **fft_xk_index** = 13 anına yerleştirilmiştir. **fft_out_valid** = 1 iken **fft_abs_sq** = 927197 ölçülmektedir. Bu değer simülasyon logundaki [Frame1 10.0MHz freqwin] xk=13 abs_sq=927197 satırı ile uyumludur. 10 MHz sinyalinin teorik frekans bin değeri 12.8 olduğundan maksimumun 13. bin’de oluşması beklenmektedir. Görselde **peak_index** hattının hâlâ 16 değerini tuttuğu görülmektedir; bu bir hata değildir. Çünkü **peak_valid** = 0 iken **peak_index** bir önceki frame’in özet sonucunu tutar. Frame 1 için yeni peak sonucu ancak frame sonunda üretilecektir.



Şekil 13: Part 3 — 10 MHz freqwin frame sonunda peak üretimi

Şekil 13’de Frame 1 tamamlandıktan sonra `peak_valid` darbesi oluşmakta ve `peak_index = 13`, `peak_value = 927197` değerleri raporlanmaktadır. Bu sonuç simülasyon logundaki `peak[1]: index=13 value=927197` satırıyla uyumludur. Bu anda `fft_xk_index` hattı artık geçerli anlık FFT bin bilgisini taşımayabilir; dolayısıyla `fft_xk_index` ile `peak_index` değerlerinin aynı anda aynı olması beklenmez. Doğru yorumlama kuralı şudur: `fft_xk_index` ve `fft_abs_sq` yalnızca `fft_out_valid = 1` iken anlamlıdır; `peak_index` ve `peak_value` ise yalnızca `peak_valid = 1` iken frame özeti olarak anlamlıdır.

Part 3 waveform görüntüleri, three-term frequency-domain windowing sonucunun Part 2’deki Hamming windowed FFT davranışıyla uyumlu olduğunu göstermektedir. 12.5 MHz frame için maksimum güç 16. bin’de, 10 MHz frame için maksimum güç 13. bin’de oluşmuştur. 12.5 MHz’de 15–17 binleri çevresinde oluşan ana lob, Hamming window etkisinin frekans domeninde doğru şekilde üretildiğini gösterir. 10 MHz’de peak 13. bin’de kalırken uzak binlerdeki güç değerleri düşük seviyededir. Bu davranış Hamming window’un leakage yan loblarını bastırma etkisiyle uyumludur.

24.12 Part 3 Timing and Valid Signal Interpretation

AXI-Stream tabanlı FFT çıkışında `fft_xk_index` ve `fft_abs_sq` yalnızca `fft_out_valid = 1` iken geçerlidir. `fft_out_valid = 0` olduğunda bu bus’lar eski veya geçersiz değerler tutabilir. Buna karşılık `peak_index` ve `peak_value`, bir frame boyunca izlenen maksimum değer frame sonunda register’a alınmış halidir ve yalnızca `peak_valid = 1` darbesi ile birlikte yorumlanmalıdır.

Bu nedenle waveform üzerinde `fft_xk_index = 13` görüldüğü anda `peak_index` değerinin hâlâ 16 olması hata değildir; çünkü Frame 1 henüz tamamlanmamış ve yeni peak sonucu üretilmemiştir. Part 3’te ek gecikme FFT IP’nin Burst I/O modunda çalışmasından kaynaklanmamaktadır. FFT IP Pipelined Streaming I/O olarak yapılandırılmıştır. Ek gecikmenin temel nedeni, `three_term.v` modülünün her output bin için komşu FFT binlerine ihtiyaç duyması ve bu nedenle önce tüm frame’i yakalayıp sonra sweep yapmasıdır.

24.13 Part 3 Conclusion

Part 3 kapsamında, Part 2’de zaman domeninde uygulanan Hamming window’un frekans domenindeki üç terimli eşdeğeri Verilog ile gerçekleştirilmiştir. Bu eşdeğerlik, FFT çıkışında

$$Y[m] = 0.54X[m] - 0.23X[m - 1] - 0.23X[m + 1]$$

işlemi ile uygulanmıştır. Simülasyon sonuçlarında 12.5 MHz için `peak_index = 16`, 10 MHz için `peak_index = 13` elde edilmiş; peak değerleri ve toplam enerji sonuçları Part 2 ile oldukça yakın çıkmıştır. Küçük farklar fixed-point katsayı yuvarlama, FFT IP scaling ve donanımsal bit genişliği tercihlerinden kaynaklanmaktadır. Bu sonuç, frequency-domain three-term windowing yönteminin time-domain Hamming windowing ile donanım üzerinde tutarlı biçimde eşdeğer sonuç verdiğini doğrulamaktadır. Testbench’te frame’lerin ayrılması, FFT IP’nin sınırlamasından değil, `three_term.v` modülünün tek-buffer capture/sweep mimarisinden kaynaklanan frame zamanlamasını doğru yönetmek için yapılmıştır.

25 Part 4: Zynq PS/PL FFT Sistemi

Part 4, önceki bölümlerde izole olarak doğrulanan FFT veri yolunun Zynq SoC mimarisi içine yerleştirilmiş halidir. Bu aşamada amaç yalnızca FFT hesaplatmak değil; Processing System (PS) tarafında çalışan bare-metal yazılımın, Programmable Logic (PL) içindeki DDS ve FFT zincirini AXI-Lite register erişimleri ile kontrol edebildiğini ve sonuçları geri okuyabildiğini göstermektir. Böylece Part 4, “PS frekansı yazar, PL sinyali üretir, FFT peak’i bulur, PS sonucu okur ve test PASS olur” akışını kapatan sistem seviyesinde doğrulama adımıdır.

Part 4 kapsamında kullanılan üst seviye kullanıcı modülü `fft_pl_ps_top.v` dosyasıdır. Bu modül; AXI-Lite slave register bankasını, DDS konfigürasyon zincirini, FFT veri yolunu, peak tespit mantığını ve debug çıkışlarını tek bir hiyerarşi altında toplar. Vivado IP Integrator tarafında bu modül `Add Module...` yaklaşımı ile block design içine `fft_pl_ps_0` adıyla eklenmiştir. Sonuçta oluşan `system.bd` tasarımı, sentez aşamasında `system_wrapper.v` üzerinden üst seviye donanım olarak derlenmektedir.

25.1 Block Design

Block Design, Vivado’nun IP Integrator aracı ile oluşturulan grafiksel sistem tarifidir. Zynq PS bloğu, reset ve AXI altyapısı ile kullanıcı RTL modülü aynı şema üzerinde bağlandığı için adresleme, saat dağıtımı, reset yönetimi ve debug bağlantıları merkezi olarak izlenebilmektedir. Bu nedenle Part 4’te block design yalnızca şematik bir görünüm değil, PS/PL entegrasyonunun doğruluğunu belirleyen temel sistem katmanıdır.

25.2 BD’deki IP Listesi

IP Bloğu	Türü	Görevi
<code>processing_system7_0</code>	ZYNQ7 PS	ARM Cortex-A9 çekirdekleri, DDR arayüzü, MIO bağlantıları ve <code>FCLK_CLK0</code> saat üretimi
<code>rst_ps7_0_200M</code>	Processor System Reset	PS kaynaklı reset bilgisinden senkron <code>peripheral_aresetn</code> üretimi
<code>axi_interconnect_0</code>	AXI Interconnect	PS tarafındaki <code>M_AXI_GP0</code> master arayüzünü PL tarafındaki AXI-Lite slave arayüzüne bağlama
<code>fft_pl_ps_0</code>	Module Reference	<code>fft_pl_ps_top.v</code> modülünün BD içindeki karşılığı; register bankası, DDS, FFT ve peak tespit zincirini içerir
<code>ila_0</code>	Integrated Logic Analyzer	Donanım üzerinde debug sinyallerinin gerçek zamanlı izlenmesi
<code>Constant</code>	Utility IP	<code>peak_index</code> sinyali için üst bitleri sıfırla dolduran sabit vektör üretimi
<code>Concat</code>	Utility IP	Sabit sıfır vektörü ile <code>peak_index_dbg[7:0]</code> sinyalini birleştirerek 32-bit debug bus oluşturma

Tablo 9: Part 4 block design içinde kullanılan temel IP ve yardımcı bloklar

Tablo 9’de görüldüğü gibi sistem hem standart Xilinx IP’lerini hem de kullanıcı tarafından yazılmış RTL modülünü birlikte kullanmaktadır. Buradaki kritik nokta, `fft_pl_ps_0` bloğunun paketlenmiş özel bir IP değil, doğrudan kullanıcı RTL’inden türetilmiş bir **Module Reference** olmasıdır. Böylece AXI-Lite arabirimi ile FFT veri yolu aynı modül içinde yönetilmekte ve PS ile PL arasındaki kontrol mantığı tek noktada toplanmaktadır.

25.3 Bağlantı Haritası

Bağlantı	Kaynak	Hedef
Saat 200 MHz	FCLK_CLK0	<code>axi_interconnect_0/{ACLK,S00_ACLK,M00_ACLK}</code> , <code>fft_pl_ps_0/clk</code>
Reset	<code>rst_ps7_0_200M/peripheral_aresetn</code>	interconnect reset girişleri ve <code>fft_pl_ps_0/rst_n</code>
AXI master	<code>processing_system7_0/M_AXI_GP0</code>	<code>axi_interconnect_0/S00_AXI</code>
AXI-Lite slave	<code>axi_interconnect_0/M00_AXI</code>	<code>fft_pl_ps_0/S_AXI</code>
Debug probeleri	<code>fft_pl_ps_0</code> çıkışları	<code>ila_0</code> prob girişleri
Dış pinler	<code>processing_system7_0</code>	DDR ve <code>FIXED_IO</code> bağlantıları

Tablo 10: Part 4 block design bağlantı haritası

Tablo 10, sistemdeki temel veri ve kontrol yollarını özetlemektedir. PS tarafındaki yazılım erişimleri doğrudan `processing_system7_0/M_AXI_GP0` master portundan başlamakta, `axi_interconnect_0` üzerinden taşınmakta ve PL tarafındaki `fft_pl_ps_0/S_AXI` slave portuna ulaşmaktadır. Böylece işlemci tarafı, PL çevrebirimini standart bir AXI-Lite adres uzayı üzerinden sürmektedir. Aynı zamanda saat ve reset bağlantılarının açıkça görünmesi, Zynq tabanlı entegrasyonun doğrulanmasını kolaylaştırmaktadır.

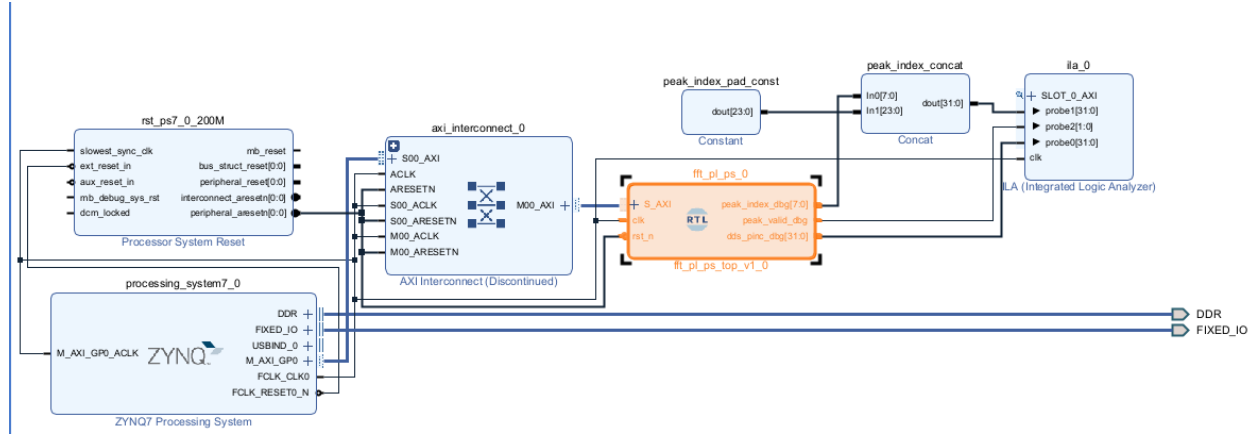
25.4 Adres Eşlemesi

Part 4 tasarımında PS tarafından erişilen PL register alanı tek bir AXI-Lite slave çevrebirimi altında toplanmıştır. Adres eşlemesi aşağıdaki gibidir:

```
Master:  processing_system7_0/Data
Slave:   fft_pl_ps_0/S_AXI / reg0
Offset:  0x43C00000
Range:   4 KB
```

Bu adres aralığı, Zynq PS’in GP0 master arayüzü üzerinden erişilebilen PL çevrebirim bölgesinin bir parçasıdır. Dolayısıyla yazılım tarafında tanımlanan taban adres üzerinden DDS faz artım sözcüğü, durum register’ları ve peak tespit sonuçları doğrudan okunup yazılabilmektedir.

25.5 Donanım Blok Diyagramı ve Debug Sinyalleri



Şekil 14: Part 4 — Vivado Block Design üzerinde Zynq PS/PL FFT sistemi ve debug bağlantıları

Şekil 14’de verilen Vivado Block Design, Part 4 tasarımının Zynq Processing System ile PL tarafındaki FFT hızlandırıcı yapısını aynı diyagram üzerinde göstermektedir. Bu yapıda PS tarafı AXI master olarak davranmakta, PL tarafındaki **fft_pl_ps_0** modülü ise AXI-Lite slave çevrebirimi olarak görev yapmaktadır. Yazılım tarafından başlatılan register erişimleri **processing_system7_0/M_AXI_GPO** portundan çıkmakta, **axi_interconnect_0** üzerinden taşınmakta ve **fft_pl_ps_0/S_AXI** portunda sonlanmaktadır. Böylece PS tarafı DDS ayar sözcüğünü, kontrol bitlerini ve sonuç okumalarını standart AXI tabanlı bir register arayüzü üzerinden gerçekleştirebilmektedir.

Diyagramdaki ana blokların görevleri kısaca şu şekildedir. **processing_system7_0** bloğu ARM çekirdeklerini, DDR arayüzünü, MIO bağlantılarını ve PL tarafına verilen saat/reset altyapısını içeren Zynq PS çekirdeğidir. **rst_ps7_0_200M** bloğu, PS kaynaklı reset bilgisini PL tarafında güvenli biçimde kullanılacak senkron reset sinyallerine dönüştüren **Processor System Reset** bloğudur. **axi_interconnect_0**, PS master arayüzü ile PL slave arayüzü arasındaki AXI kanal eşlemesini ve bağlantı yönlendirmesini üstlenmektedir. **fft_pl_ps_0** bloğu ise AXI-Lite register bankası, DDS kontrol zinciri, FFT veri yolu ve peak tespit mantığını tek modül altında toplayan özel PL tasarımıdır. **ila_0** bloğu, sistem içindeki debug sinyallerini donanım üzerinde gerçek zamanlı gözlemlemek için eklenen Integrated Logic Analyzer birimidir. **Constant** ve **Concat** blokları ise özellikle debug bus genişliklerini ILA açısından daha okunabilir biçime getirmek amacıyla kullanılmış yardımcı bloklardır.

Saat ve reset dağıtımı bu block design’ın kritik noktalarından biridir. **processing_system7_0** tarafından üretilen **FCLK_CLK0 = 200 MHz** saati hem **axi_interconnect_0** bloğunun saat girişlerine hem de PL FFT sistemini içeren **fft_pl_ps_0** bloğuna ortak saat olarak uygulanmıştır. Bu sayede kontrol yolu ile işleme yolu aynı saat alanında tutulmuş ve ek bir clock-domain crossing ihtiyacı doğmamıştır. Reset tarafında ise **FCLK_RESETO_N** hattı önce **rst_ps7_0_200M** bloğuna verilmiş, bu blok da senkron **peripheral_asetn** hattını üreterek hem interconnect yapısına hem de PL tarafındaki çevrebirim reset girişlerine dağıtmıştır.

25.6 AXI-Lite Register Map

PS/PL haberleşmesinin özü register map üzerinde kuruludur. PS tarafı frekans isteğini DDS_PINC register'ına yazar; PL tarafı FFT sonucunu PEAK_STATUS ve PEAK_VALUE register'ları üzerinden geri verir. Sticky valid bit'inin kullanılma nedeni, FFT pipeline'ının ARM yazılımından çok daha hızlı çalışması ve “yeni sonuç hazır” bilgisinin yazılım tarafından kaçırılmadan okunmasını sağlamaktır.

Offset	Register	Erişim	Açıklama
0x00	DDS_PINC	RW	PS tarafından yazılan 32-bit DDS phase increment değeri
0x04	PEAK_STATUS	RO / W1C bit 8	[7:0] peak index, [8] peak_valid_sticky
0x08	PEAK_VALUE	RO	Tespit edilen peak büyüklük-kare değeri; [24:0] alanı 32-bit'e sıfır genişletilir
0x0C	BUILD_INFO	RO	Sabit 0x0004_0001 versiyon/imza bilgisi

Tablo 11: Part 4 AXI-Lite register map

Bu tabloda özellikle PEAK_STATUS register'ı kritik önemdedir. **peak_valid_sticky** biti bir kez yükseldiğinde yazılım tarafından write-1-clear işlemi ile temizlenene kadar 1 kalır. Böylece PL tarafındaki kısa süreli geçerli işareti, PS tarafındaki daha yavaş polling döngüsü ile güvenilir biçimde yakalanabilmektedir.

25.7 DDS PINC Hesabı ve Frekans Kontrolü

Part 4'te istenen çıkış frekansı doğrudan frekans biriminde değil, DDS faz akümülatörünün PINC değeri üzerinden PL tarafına gönderilmektedir. W-bit genişliğinde bir DDS faz akümülatöründe üretilen frekans

$$f_{\text{out}} = \frac{\text{PINC}}{2^W} f_s \quad (49)$$

şeklindeir. Bu projede DDS faz genişliği $W = 32$ bit ve sistem saati $f_s = 200$ MHz olduğundan, istenen frekansa karşılık gelen register değeri

$$\text{PINC} = \text{round}\left(\frac{f_{\text{desired}}}{f_s} 2^{32}\right) \quad (50)$$

ile hesaplanır.

Örneğin 10 MHz için:

$$\text{PINC} = \text{round}\left(\frac{10 \times 10^6}{200 \times 10^6} 2^{32}\right) = 214748365 = 0x0CCCCCD \quad (51)$$

Aynı hesapla 12.5 MHz için 0x10000000, 25 MHz için 0x20000000 ve 50 MHz için 0x40000000 elde edilir. Bu bölüm, PS tarafından yazılan ham sayının PL içindeki gerçek çıkış frekansına nasıl dönüştüğünü açıkça göstermektedir.

25.8 FFT Peak Readback Mantığı

PL tarafında DDS ile üretilen sinyal FFT zincirinden geçirilir ve her frame sonunda en büyük $|X[k]|^2$ değerine ait bin indeksi ile büyüklük-kare değeri register alanına yazılır. Yazılım tarafı için temel okuma mantığı şöyledir: önce sticky valid biti temizlenir, ardından yeni bir frame sonucunun hazır olması beklenir, sonrasında PEAK_STATUS ve PEAK_VALUE register'ları okunur.

FFT bin çözünürlüğü Part 4'te de

$$\Delta f = \frac{f_s}{N} = \frac{200 \text{ MHz}}{256} = 781.25 \text{ kHz} \quad (52)$$

olarak kalmaktadır. Yazılım tarafında okunan peak indeksinden frekans tahmini genel biçimde

$$k_{\text{signed}} = \begin{cases} k, & k < N/2 \\ k - N, & k \geq N/2 \end{cases} \quad (53)$$

$$f_{\text{est}} = k_{\text{signed}} \frac{f_s}{N} \quad (54)$$

ile hesaplanabilir. Bu raporda ölçülen frekanslar pozitif yarı bantta yer aldığı için testlerde doğrudan peak_index değeri kullanılmıştır. Karşılaştırma kriteri ise $|f_{\text{target}} - f_{\text{est}}| \leq f_s/N$ yani bir FFT bininden küçük hata payıdır.

25.9 ILA Debug Sinyalleri

Debug altyapısı, tasarımın fonksiyonel veri yolunun zorunlu bir parçası değildir; yalnızca donanın üzerinde gözlem ve doğrulama amacıyla eklenmiştir. Bu kapsamda fft_pl_ps_0 bloğundan çıkarılan peak_index_dbg[7:0], peak_valid_dbg ve dds_pinc_dbg[31:0] sinyalleri ILA'ya bağlanmıştır. dds_pinc_dbg sinyali PS tarafından yazılan DDS faz artım sözcüğünün PL içine doğru geçtiğini izlemek için, peak_valid_dbg sinyali bir frame sonucunun hazır olduğunu görmek için ve peak_index_dbg sinyali ise tespit edilen baskın FFT binini doğrudan gözlemlemek için kullanılmıştır.

peak_index_dbg sinyali yalnızca 8 bit genişliğinde olduğu için, ILA ekranında diğer 32-bit debug bus'ları ile birlikte daha okunabilir görünmesi amacıyla üst bitler sıfır ile doldurulmuştur. Bu amaçla alt 8 bit doğrudan peak_index_dbg[7:0] olarak kullanılmış, üst 24 bit ise peak_index_pad_const[23:0] sabit sıfır vektörü ile üretilmiştir. Ardından peak_index_concat[31:0] = {24'd0, peak_index_dbg[7:0]} eşdeğerinde bir birleştirme yapısı Constant ve Concat bloklarıyla oluşturularak ILA prob girişine verilmiştir. Bu düzenleme yalnızca debug okunabilirliğini artırmakta, tasarımın hesaplama sonucunu veya veri yolunun işlevini değiştirmemektedir.

Üst seviye debug port yapısı aşağıdaki kısa Verilog parçasında özetlenmiştir:

```
output wire [7:0] peak_index_dbg,  
output wire      peak_valid_dbg,  
output wire [31:0] dds_pinc_dbg
```

ILA tarafında kullanılan sıfır genişletme mantığı ise aşağıdaki kısa yapı ile ifade edilebilir:

```
wire [23:0] peak_index_pad_const;  
wire [31:0] peak_index_concat;  
  
assign peak_index_pad_const = 24'd0;  
assign peak_index_concat    = {peak_index_pad_const, peak_index_dbg};
```

25.10 Vitis Bare-Metal Test Yazılımı

Part 4'ün sistem seviyesinde doğrulanması için Vitis tarafında kısa bir bare-metal C uygulaması kullanılmıştır. Test akışı şu sırayı izlemektedir:

- Hedef frekans için PINC değeri hesaplanır.
- Değer DDS_PINC register'ına yazılır.
- PEAK_STATUS içindeki sticky valid biti temizlenir.
- Yeni peak sonucunun hazır olması beklenir.
- PEAK_STATUS ve PEAK_VALUE okunur.
- peak_index değeri frekansa çevrilir.
- Hata bir FFT bininden küçükse test PASS kabul edilir.

Bu akışın özü aşağıdaki kısa C parçasında özetlenebilir:

```
write_reg(DDS_PINC_OFFSET, pinc);  
write_reg(PEAK_STATUS_OFFSET, 0x100); // sticky valid clear  
  
do {  
    status = read_reg(PEAK_STATUS_OFFSET);  
} while ((status & 0x100) == 0);  
  
peak_idx    = status & 0xFF;  
peak_value = read_reg(PEAK_VALUE_OFFSET);
```

Bu yazılım yaklaşımı ile PS tarafı, PL içindeki tüm veri yolunu yalnızca birkaç register erişimi üzerinden kontrol edebilmektedir. Böylece Part 4, donanım hızlandırıcının salt bir blok olmaktan çıkıp yazılım tarafından sürülebilen gerçek bir sistem bileşeni haline geldiğini göstermektedir.

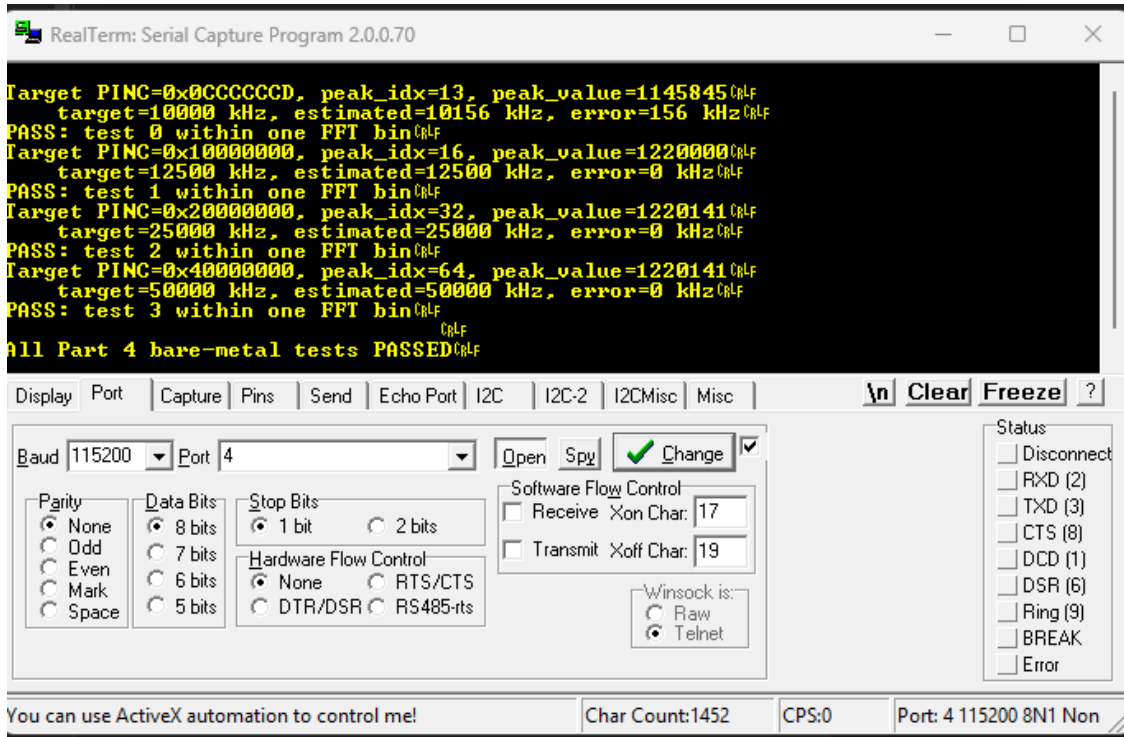
25.11 Donanım Üzerinde Ölçülen Sonuçlar

Vitis bare-metal uygulaması Zybo Z7-20 üzerinde JTAG aracılığıyla çalıştırılmış; UART çıkışı RealTerm üzerinden 115200 8N1 yapılandırması ile yakalanmıştır. Aşağıdaki tablo, dört test frekansı için ölçülen `peak_index`, tahmin edilen frekans ve `peak_value` sonuçlarını özetlemektedir.

Hedef Frekans	PINC (hex)	Peak Index	Tahmin	Hata	Peak Value	Sonuç
10 MHz	0x0CCCCCD	13	10.156 MHz	156 kHz	1 144 900	PASS
12.5 MHz	0x10000000	16	12.5 MHz	0 kHz	1 219 301	PASS
25 MHz	0x20000000	32	25 MHz	0 kHz	1 222 065	PASS
50 MHz	0x40000000	64	50 MHz	0 kHz	1 220 785	PASS

Tablo 12: Part 4 donanım üzerinde ölçülen frekans tespit sonuçları

Tablo 12, Part 4'ün donanım üzerinde gerçekten çalıştığını gösteren temel kapanış bölümüdür. Dört testin tamamı, $f_s/N = 781.25 \text{ kHz}$ olan FFT bin çözünürlüğünün altında hata ile PASS olmuştur. 12.5 MHz, 25 MHz ve 50 MHz frekansları FFT bin merkezlerine tam oturduğu için hata sıfırdır. 10 MHz testindeki 156 kHz hata ise beklenen fiziksel bir davranıştır; çünkü bu frekans teorik olarak bin 12.8'e karşılık gelmekte ve enerji bin 12 ile bin 13 arasında paylaşılmaktadır. Buna rağmen peak tespiti doğru biçimde bin 13 üzerinde yoğunlaşmış ve hata bir FFT bininin oldukça altında kalmıştır.



Şekil 15: Part 4 — RealTerm üzerinden alınan UART/serial terminal çıktısı

Şekil 15, Vitis bare-metal uygulamasının kart üzerinde çalıştırılması sonrası RealTerm üzerinden

gözlendi. UART çıktısını göstermektedir. Bu görsel, PS tarafında yazılan PINC değerlerine karşılık PL tarafının peak tespit sonucunu başarıyla geri döndürdüğünü ve testlerin kart üzerinde PASS ile tamamlandığını doğrudan belgelemektedir.

UART çıktısının rapora girmesi gereken özeti aşağıdaki gibidir:

Part 4 Vitis bare-metal DDS/FFT test

Base address: 0x43c00000

Frequency resolution: 781.25 kHz

Target PINC=0x0CCCCCD, peak_idx=13, peak_value=1144900

PASS: test 0 within one FFT bin

Target PINC=0x10000000, peak_idx=16, peak_value=1219301

PASS: test 1 within one FFT bin

Target PINC=0x20000000, peak_idx=32, peak_value=1222065

PASS: test 2 within one FFT bin

Target PINC=0x40000000, peak_idx=64, peak_value=1220785

PASS: test 3 within one FFT bin

All Part 4 bare-metal tests PASSED

Bu kısa çıktı, PS tarafında hesaplanan PINC değerinin gerçekten PL içine yazıldığını, FFT sonucunun geri okunduğunu ve kart üzerinde yapılan testlerin başarıyla tamamlandığını göstermektedir. Raporda tüm log'u vermek yerine yalnızca özet satırların kullanılması, metni gereksiz yere uzatmadan donanım doğrulamasını açık biçimde belgelemektedir.

25.12 Part 4 Sonuç

Part 4 sonucunda, Zynq PS ile PL tarafındaki DDS+FFT veri yolu tek bir sistem altında başarıyla bütünleştirilmiştir. PS tarafı DDS_PINC register'ına yazarak istenen frekansı belirlemiş, PL tarafı bu frekansta sinyal üretmiş, FFT peak tespit zinciri baskın bin ve peak değerini hesaplamış, ardından sonuçlar PEAK_STATUS ve PEAK_VALUE register'ları üzerinden tekrar PS tarafına okunmuştur. Block design, ILA debug bağlantıları, register map, PINC matematiği, bare-metal test akışı ve UART tabanlı donanım sonuçları birlikte değerlendirildiğinde Part 4, tasarımın yalnızca simülasyonda değil gerçek kart üzerinde de kapalı çevrim biçimde doğrulandığını göstermektedir. Bu sonuç, Part 1–Part 3'te simülasyon ortamında doğrulanan FFT ve windowing veri yolunun, Part 4'te PS tarafından kontrol edilebilen gerçek bir Zynq çevrebirimine dönüştürüldüğünü göstermektedir.

26 Kaynakça ve Ekler

26.1 Kaynakça

Aşağıdaki tablo, proje boyunca yararlanılan temel teknik kaynakları ve bu kaynakların kullanım amaçlarını özetlemektedir.

No	Kaynak	Kullanım Amacı	Bağlantı / Not
1	Baryonic Space FPGA/Digital Design Engineer Interview Project	Projenin ana görev tanımı, Part 1–Part 4 kapsamı, FFT/windowing beklentileri, Zynq PS/PL bonus isterleri ve doğrulama hedefleri için temel referans olarak kullanılmıştır.	Kullanıcıya sağlanan proje PDF dokümanı.
2	Xilinx FFT IP / Vivado Generated Instantiation	Xilinx FFT IP’nin 256-point, fixed-point, AXI-Stream tabanlı kullanımı; tdata/tuser portları, xk_index çıkışı, config kanalı ve natural-order FFT çıkışı için referans alınmıştır.	Vivado tarafından üretilen xfft_0 IP wrapper/instantiation dosyaları ve IP configuration ekranları.
3	AMBA AXI4-Stream Protocol Specification	AXI-Stream valid/ready handshake, tdata , tlast ve tuser sinyallerinin yorumlanması için temel protokol referansı olarak kullanılmıştır.	Scribd üzerinde paylaşılan AMBA AXI4-Stream v1.0 protokol dokümanı.
4	Mehmet Burak Aykenar YouTube Channel	Vivado, FPGA tasarım akışı, Zynq SoC, AXI tabanlı özel IP oluşturma ve donanım/yazılım entegrasyonu konularında genel öğrenme ve uygulama desteği için kullanılmıştır.	YouTube kanalındaki Vivado ve Zynq odaklı eğitim içerikleri.
5	Mehmet Burak Aykenar GitHub Repository — apis_anatolia	AXI-Lite/custom IP örnekleri, Vivado IP paketleme yaklaşımı, register tabanlı çevreirim mantığı ve test/verification örnekleri için referans alınmıştır. Özellikle AXI4-Lite slave register mantığı ve custom IP yapısının anlaşılmasında faydalanılmıştır.	GitHub deposu: mbaykenar/apis_anatolia .
6	Mehmet Burak Aykenar — Custom AXI4-Lite / AXI4 IP Examples	Vivado’da AXI4-Lite arayüzlü custom IP oluşturma, register bankası mantığı ve AXI-Lite tabanlı PS/PL haberleşme yapısını anlamak için kullanılmıştır.	İlgili blog yazısı ile axifull_ip ve axilite_uvvm örnek klasörleri.
7	Farbius DSP Xilinx IP GitHub Repository	Xilinx IP çekirdekleriyle DSP uygulamaları, FFT IP kullanımı ve proje kapsamındaki genel DSP-on-FPGA yaklaşımı için referans alınmıştır.	Farbius tarafından paylaşılan DSP-on-FPGA örnek deposu.

No	Kaynak	Kullanım Amacı	Bağlantı / Not
8	Embedded.com — DSP Tricks: Frequency-Domain Windowing	Part 3'te kullanılan Hamming window'un frekans domenindeki üç terimli eşdeğerinin teorik temeli için kullanılmıştır.	https://www.embedded.com/dsp-tricks-frequency-domain-windowing/
9	Richard G. Lyons — <i>Understanding Digital Signal Processing</i>	Frequency-domain windowing kavramı ve three-term windowing eşdeğerliği için teorik arka plan olarak kullanılmıştır.	Embedded.com'daki frequency-domain windowing yazısı, <i>Understanding Digital Signal Processing</i> kitabındaki ilgili DSP tricks içeriğine dayanmaktadır.
10	Python / NumPy Reference Model	Kompleks ton üretimi, FFT referans hesapları, Hamming window katsayı üretimi ve fixed-point .mem dosyalarının oluşturulması için kullanılmıştır.	Proje kapsamında yazılan reference_fft.py ve generate_hamming_mem.py dosyaları.
11	Vivado, Vitis ve RealTerm Araçları	Vivado sentez/simülasyon/IP entegrasyonu, Vitis bare-metal PS yazılımı ve RealTerm üzerinden UART çıktısının doğrulanması için kullanılmıştır.	Kullanılan geliştirme ve doğrulama araçları.

26.2 Ekler

Aşağıdaki tablo, rapor kapsamında kullanılan temel dosya, çıktı ve yardımcı içerikleri özetlemektedir.

Ek No	İçerik	Açıklama
Ek A	Python Referans Dosyaları	reference_fft.py ile 12.5 MHz ve 10 MHz kompleks tonlar üretilmiş, Python FFT ile beklenen peak bin değerleri doğrulanmıştır. generate_hamming_mem.py ile Hamming katsayıları Q1.15 formatında hamming_q1_15.mem dosyasına yazılmıştır.
Ek B	Verilog RTL Dosyaları	fft_naive_top.v , fft_windowed_top.v , three_term.v , hamming_window.v , abs_sq.v , peak_detector.v ve fft_pl_ps_top.v dosyaları tasarımın temel RTL modüllerini oluşturmaktadır.
Ek C	SystemVerilog Testbench Dosyaları	tb_fft_naive.sv , tb_fft_windowed.sv ve tb_fft_freqwin.sv dosyaları Part 1, Part 2 ve Part 3 simülasyon doğrulamalarında kullanılmıştır.
Ek D	Bellek Dosyaları	tone_12p5MHz_re.mem , tone_12p5MHz_im.mem , tone_10MHz_re.mem , tone_10MHz_im.mem ve hamming_q1_15.mem dosyaları testbench ve RTL simülasyonlarında kullanılmıştır.
Ek E	Vivado Block Design	Part 4 kapsamında Zynq PS, AXI Interconnect, fft_pl_ps_0 module reference, ILA, Constant ve Concat bloklarını içeren system.bd tasarımı kullanılmıştır.
Ek F	Vitis Bare-Metal Yazılımı	PS tarafından DDS_PINC register'ına frekans ayarı yazan, PEAK_STATUS ve PEAK_VALUE register'larını okuyarak FFT peak sonucunu doğrulayan C kodu kullanılmıştır.
Ek G	UART / RealTerm Donanım Çıktısı	Zybo Z7-20 üzerinde çalıştırılan Vitis uygulamasının 10 MHz, 12.5 MHz, 25 MHz ve 50 MHz testlerini PASS verdiğini gösteren RealTerm çıktısı rapora eklenmiştir.
Ek H	Waveform Görselleri	Part 1, Part 2 ve Part 3 için fft_out_valid , fft_xk_index , fft_abs_sq , peak_valid ve peak_index sinyallerini gösteren Vivado waveform ekran görüntüleri kullanılmıştır.

26.3 AI Kullanım Beyanı

Bu raporun hazırlanması sürecinde yapay zekâ destekli araçlardan yardımcı kaynak olarak faydalanılmıştır. AI desteği; Verilog/SystemVerilog kodlarının okunabilirliğinin artırılması, hata ayıklama sırasında olası nedenlerin tartışılması, Vitis/Vivado kullanım adımlarının düzenlenmesi, rapor dilinin sadeleştirilmesi, teknik açıklamaların rapor formatına dönüştürülmesi ve tablo/başlık yapısının iyileştirilmesi amacıyla kullanılmıştır.

AI araçları, tasarım kararlarının tek kaynağı olarak kullanılmamıştır. FFT IP yapılandırması, test-bench senaryoları, simülasyon sonuçları, waveform yorumları, AXI-Lite register map, Vitis bare-metal testleri ve donanım çıktıları kullanıcı tarafından Vivado, Vitis ve kart üzerinde doğrulanmıştır. Rapor içindeki sayısal sonuçlar, simülasyon logları, waveform ekran görüntüleri ve UART çıktıları proje kapsamında elde edilen gerçek doğrulama verilerine dayanmaktadır.

AI kullanımı özellikle şu alanlarda sınırlı kalmıştır:

- Kod bloklarının daha okunabilir hale getirilmesi
- Hata nedenlerinin yorumlanması
- AXI-Stream ve AXI-Lite sinyal açıklamalarının sadeleştirilmesi
- Matematiksel ifadelerin rapor formatına dönüştürülmesi
- Raporun bölüm yapısı, tablo düzeni ve sonuç yorumlarının iyileştirilmesi
- Türkçe teknik metnin daha anlaşılır ve tutarlı hale getirilmesi

Sonuç olarak AI, projede yardımcı bir düzenleme ve açıklama aracı olarak kullanılmış; tasarımın fonksiyonel doğrulaması ise simülasyon, waveform analizi ve Zybo Z7-20 üzerinde alınan gerçek UART çıktıları ile yapılmıştır. Bu nedenle rapordaki teknik sonuçların sorumluluğu kullanıcıya ait olup, AI yalnızca destekleyici düzenleme ve açıklama aracı olarak kullanılmıştır.